

Extended OBU Library (SDK) Developer Guide

Third Party Application for Android & iOS

Revision 1.6
27 July 2026

Copyright © 2026 by Land Transport Authority. All rights reserved.

All trademarks or registered trademarks mentioned in this document are properties of their respective owners.

No part of this document may be reproduced or transmitted in any form or by any means, electronic, mechanical or otherwise, without a prior written permission.

If you are not the intended recipient, you are hereby notified that any disclosure, copying, distribution, or the taking of any action based on the contents of this information is strictly prohibited. If you received this document by error, you are kindly requested to immediately contact us.

Revision History

Revision	Effective Date	Description
1.0	01 Mar 2023	First Release
1.1	01 May 2023	Minor Revision
1.2	30 Jun 2023	Updated Sections: - Updated Section 1.3 Description - Updated Section 3.8.3.1 Code Snippet and Description - Updated Section 3.8.3.5 Function List Table - Updated Section 3.8.4 to add Function for item 10 - Added Section 3.8.6 Traffic Icons in SDK for Android - Updated Section 4.7 Code Snippet - Updated Section 4.7.6 Function List Table - Added Section 4.7.8 Traffic Icons in SDK for iOS - Updated Section 5.4 to remove content4 from Function List Table - Updated Section 5.7 Parameter Names - Moved Section 6.8.11 OBUDData to Section 6.8 - Moved Section 6.8 Enums to Section 6.9 - Added Appendix A for TDCID icons
1.3	29 Sep 2023	Updated “nexgen ERP” to “ERP 2.0” Updated Sections: - Update Section 3.5 Code Snippet - Update Section 3.8.3.1 to note of using location services - Update Section 3.8.3.5, item 5 - Travel Summary description - Update Section 4.7.5.2, item 7 - Travel summary description - Update Section 4.7.5.2, added note on item 8 – TotalTripCharged
1.4	27 Dec 2024	Added the word “Bluetooth” to MAC Address Updated Sections: - Updated section 3.5 and 4.5 code snippet with Github URL - Updated section 3.8.3.2 and 4.7.4 to explain the use of vehicle number for connecting to OBU - Updated section 3.8.3.5 and 4.7.5.2 item 6 – onPaymentHistorise description - Deleted section 3.8.4, item 6 – getLastOBUDData - Updated section 3.8.5 and 4.7.7 Mock Manager - Updated section 5.1 to 5.3 to indicate configurable and non-configurable items and OBU sample screens - Deleted section 6.3, roadName and cardBalance - Updated section 6.9.3, rename “Common” to “DetectAlertPointCommonAlert”
1.5	01 Sep 2025	Updated Figure: - Updated Figure 7 – Travel Time Information - Updated Figure 12 – ERP Charging Screen in OBU - Updated Figure 16 – EPS Charging Screen in OBU - Updated Figure 18 – EEP Charging Screen in OBU Updated Sections: - Updated section 3.8.3.2 Description of connect to an OBU - Updated section 4.7.3.2 Description of connect to an OBU

		- Updated Appendix A for additional Picto IDs for Travel Time Display message icons.
1.6	27 Jul 2026	Updated Sections: - Added section 1.1 Understanding business functions - Added section 1.9 Roadside Electronic Parking - Added section 1.10 Off-Peak Car - Added section 1.11 Check Point Tolling - Added description to section 2 for indicating SDK versions covered by this developer guide - Updated section 2.2 to provide EXTOL's GitHub's URL - Updated section 3.8.3.5 Subscribe to the OBUDataListener - Added section 3.8.3.6 ERPMessage Callback - Added section 3.8.3.7 ERPMessage Types - Added section 3.8.3.8 Migration from OBUChargingInformation to ERPMessage - Added section 3.8.3.9 Code Migration Examples - Updated section 3.8.5 Mock Manager for Android Applications - Removed section 4.4 for Third-Party Libraries and replaced with adding SDK to iOS applications - Added section 4.4.2 Swift Package Manager details - Updated section 4.6.2 Description of OBUDataPermission - Updated section 4.6.5.2 Description of OBU Data Delegate, point 3 and 5 in the table - Added section 4.6.6 ERPMessage callback - Added section 4.6.7 ERPMessage types - Added section 4.6.8 Migration from OBUChargingInformation to ERPMessage - Added section 4.6.9 Code Migration examples - Updated section 4.6.11 Mock Manager for iOS applications - Removed ERP, EEP, EPS details from section 5 - Removed OBUChargingInformation and OBU Charging Message Type class from section 6 - Updated section 6.3 OBU Payment History class details - Updated section 6.7 OBU Data class details - Updated section 6.8.2 OBUBusinessFunction class details - Updated section 6.8.4 OBUChargingType class details - Updated section 6.8.7 OBUTextStyle class details - Updated section 6.8.8 OBUTrafficDataPriority class details

Table of Contents

1	Overview	10
1.1	Understanding Business Functions	10
1.2	Electronic Road Pricing	11
1.3	Point-Based Charging	12
1.4	Electronic Parking System.....	13
1.5	Enhanced Electronic Parking	13
1.6	Common Alert	14
1.7	REP (Roadside Electronic Parking).....	15
1.8	OPC (Off-Peak Car)	15
1.9	CPT (Check Point Tolling).....	16
1.10	Traffic Information (TDCID)	17
2.	Getting Started	19
2.1	Getting SDK Account Key/Sign Up	19
2.2	Sample Application with SDK Integration.....	19
2.3	Register Your Phone at e-Services Portal	20
2.3.1	Bluetooth MAC Address Authentication for Android Phones.....	20
2.3.2	Bluetooth MAC Address Authentication for iPhones	20
3.	Android.....	21
3.1	Minimum API Level for the SDK.....	21
3.2	Programming Language	21
3.3	Bluetooth Requirement.....	21
3.4	Third-Party Libraries.....	21
3.5	How to Add the SDK to Android Application.....	21
3.6	SDK Permissions	22
3.6.1	SDK Custom Permission for Third-Party Applications	22
3.7	Other Requirements	23
3.8	How to Integrate the SDK.....	23
3.8.1	Step 1: Initialise the SDK with an SDK Account Key.....	23
3.8.2	Step 2: OBU Data Permission.....	24
3.8.3	Step 3: Establish a Connection to the OBU	24
3.8.4	SDK Helper Methods for OBU Data	44

3.8.5 Mock Manager for Android Applications	45
3.8.6 Traffic Icons in the OBU SDK.....	50
4. IOS	50
4.1 Minimum iOS Version	50
4.2 Programming Language	50
4.3 Bluetooth Requirement	50
4.4 How to Add the SDK to iOS Applications	51
4.4.1 CocoaPods	51
4.4.2 Swift Package Manager (SPM).....	51
4.5 SDK Permissions	52
4.6 How to Integrate the SDK	52
4.6.1 Step 1: Initialise SDK with the SDK Account Key.....	52
4.6.2 Step 2: OBU Data Permission.....	53
4.6.3 Step 3: Search for OBUs.....	53
4.6.4 Connect to an OBU.....	54
4.6.5 Connection Status Delegate.....	54
4.6.6 ERPMessage Callback.....	58
4.6.7 ERPMessage Types.....	60
4.6.8 Migration Guide from OBUChargingInformation to ERPMessage....	66
4.6.9 Code Migration Examples	68
4.6.10 SDK Helper Methods for OBU Data	69
4.6.11 Mock Manager for iOS Applications	71
4.6.12 Traffic Icons in the OBU SDK.....	75
5 Mock Manager for Testing Applications	76
5.1 Traffic Messages.....	76
5.1.1 Template1	76
5.1.2 Template2.....	78
5.1.3 Template3.....	78
5.1.4 Template4	79
5.1.5 Template5.....	79
5.1.6 Template6.....	79
5.1.7 Template8.....	80
5.1.8 Template110	81

5.1.9 Template111	81
5.1.10 Template12	82
5.1.11 Template13	83
6 API Classes.....	84
6.1 OBU	84
6.2 OBUAcceleration	84
6.3 OBUPaymentHistory	84
6.4 OBUTravelSummary	85
6.5 OBUTotalTripCharged.....	86
6.6 Traffic Information	86
6.6.1 Usage in Android.....	86
6.6.2 Usage in iOS	87
6.7 OBUData	88
6.8 Enums.....	89
6.8.1 OBUCardStatus.....	90
6.8.2 OBUBusinessFunction.....	90
6.8.3 OBUCargingPaymentMode.....	90
6.8.4 OBUCargingType	91
6.8.5 OBUPaymentMode.....	91
6.8.6 OBUTextColor.....	91
6.8.7 OBUTextStyle	92
6.8.8 OBUTrafficDataPriority	92
6.8.9 OBUTrafficMessageType.....	93
6.8.10 BluetoothState (iOS)	93
6.8.11 BluetoothState (Android).....	94
6.8.12 OBUErrorCode	94
7 SDK Troubleshooting	97
8 Appendix A - TDCID Icons	98

Table of Figures

Figure 1-1 EXTOL SDK Flow Overview	10
Figure 1-2 Point Based Charging.....	12
Figure 1-3 Electronic Parking System.....	13
Figure 1-4 Enhanced Electronic Parking	14
Figure 1-5 Common Alert Display	14
Figure 1-6 Roadside Electronic Parking	15
Figure 1-7 Off-Peak Car	15
Figure 1-8 Check Point Tolling	16
Figure 1-9 General Traffic Information (School Zone)	17
Figure 1-10 Travel Time Information	17
Figure 1-11 Parking Lot Information	18
Figure 1-12 Developer Workflow Overview	19
Figure 1-13 Bluetooth MAC Address Input Screen for iOS.....	20
Figure 1-14 OBU Search Screen	26

Abbreviations

Abbreviation	Definition
APK	Android Package
API	Application Programming Interface
AZ	Alert Zone
CDZ	Charging Display Zone
CPO	Car Parking Owner
CPT	Check Point Tolling
EEP	Enhanced Electronic Parking
ERP	Electronic Road Pricing
ERP1	Current Electronic Road Pricing system with gantries
IU	In-vehicle Unit
REP	Roadside Electronic Parking
OBU	On Board Unit
OPC	Off-Peak Car
SDK	Extended OBU Library (EXTOL) Software Development Kit
TDCID	Traffic Data Collection and Information Dissemination

1 OVERVIEW

The Extended OBU Library (EXTOL SDK) aims to enhance your applications by providing APIs to connect and receive ERP (Electronic Road Pricing) and Traffic messages from the OBU while driving on the roads to help improve the driving experience. It is intended for third-party developers and companies who want to integrate OBU Display messages into their applications, enabling them to serve as an alternative display for motorists or to use the data from the NexGen ERP OBU for innovative applications. The SDK provides high level APIs that make integration into your application swift and straightforward. Communication between the SDK and OBU device is accomplished via Bluetooth.

This guide covers all the details required for integration into your applications for Android and iOS platforms. The diagram below explains the high-level communication between the devices:



Figure 1-1 EXTOL SDK Flow Overview

Note: OBU can only connect to one SDK at a time.

1.1 Understanding Business Functions

The OBU SDK supports the following charging business functions that represent different types of road charging, parking services, and traffic information in Singapore. Understanding these business functions is essential before implementing the SDK callbacks.

Charging Business Functions:

- ERP (Electronic Road Pricing) that includes Point-Based charging type for the new ERP system. It also includes ERP1, which refers to the existing (legacy) ERP charging type.
- EEP (Enhanced Electronic Parking)
- EPS (Electronic Parking System)
- REP (Road-side Electronic Parking)
- OPC (Off-Peak Car)
- CPT (Check Point Tolling)

Each message received from the SDK is associated with one of these business functions. Each Business functions and charging types is explained in the next sections.

Traffic Information (TDCID):

TDCID (Traffic Data Collection and Information Dissemination) provides real-time traffic information. TDCID can be received either as a standalone callback or combined with ERP messages in a single callback.

1.2 Electronic Road Pricing

In the Next Generation Electronic Road Pricing System (ERP 2.0), the roads are divided into virtual segments such as:

- AZ (Alert Zone):** This informs the motorist that there will be charges incurred ahead of the zone.
- CDZ (Charging Display Zone):** This provides the charging zone information. The OBU sends two back-to-back messages at this point:
 - Charging Information:** the amount that will be deducted at the CDZ.
 - Deduction Success/Failure:** the actual deducted amount at the CDZ for successful and unsuccessful deduction.

1.3 Point-Based Charging

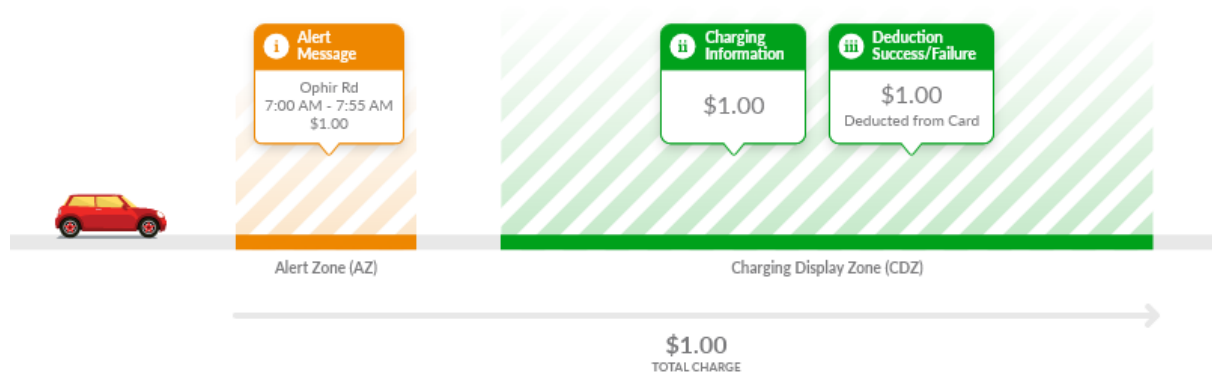


Figure 1-2 Point Based Charging

In Point-Based charging, charging takes place at a certain point on the road and the OBU sends the following messages:

- i. **Point-Based Alert:** This message informs the motorist on the charges that will be incurred during a specific timeframe at the CDZ. It displays the road name, time range and road pricing charge.
- ii. **Point-Based Charging Information:** This message displays the amount that will be deducted at the CDZ.
- iii. **Point-Based Deduction Successful/Failure:** This message displays the actual deducted amount at the CDZ. The message contains information regarding successful and unsuccessful deductions.

1.4 Electronic Parking System

Electronic Parking System enables vehicles with In-vehicle Unit (IU) to gain entry to and exit car parks. Parking charges are charged based on season parking pass or through the CashCard/ CEPAS Card that is inserted in the vehicles' IU. In ERP 2.0 system, the current In-vehicle Unit (IU) is replaced with the OBU that displays the charging information and deduction if vehicles do not have the season parking pass.

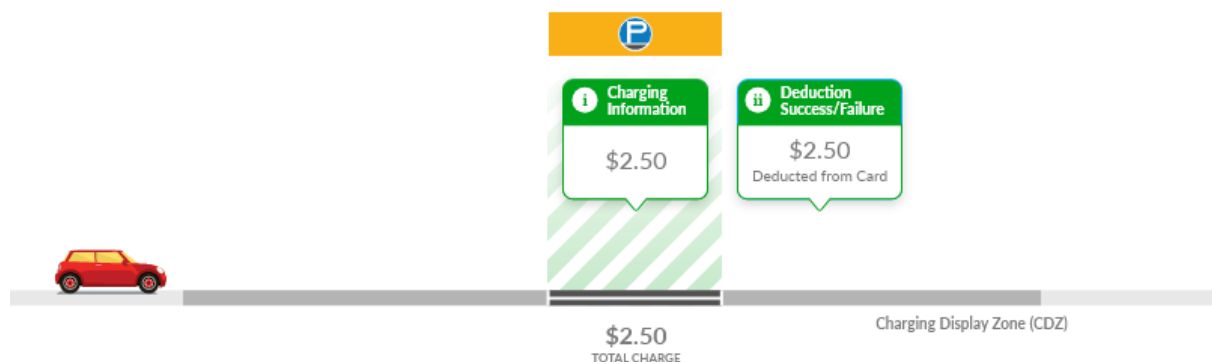


Figure 1-3 Electronic Parking System

- i. **Entry-Exit Charging Information:** This informs the motorist of the amount that will be deducted upon entering the parking area.
- ii. **Entry-Exit Deduction Successful/Failure:** This message displays the actual deducted amount at the exit point of a parking area when successful, and the expected deduction amount when unsuccessful.

1.5 Enhanced Electronic Parking

Enhanced Electronic Parking System is the enhancement of Electronic Parking System. Unlike in EPS system, EEP has CPO (Car Parking Owner) information and charging information.

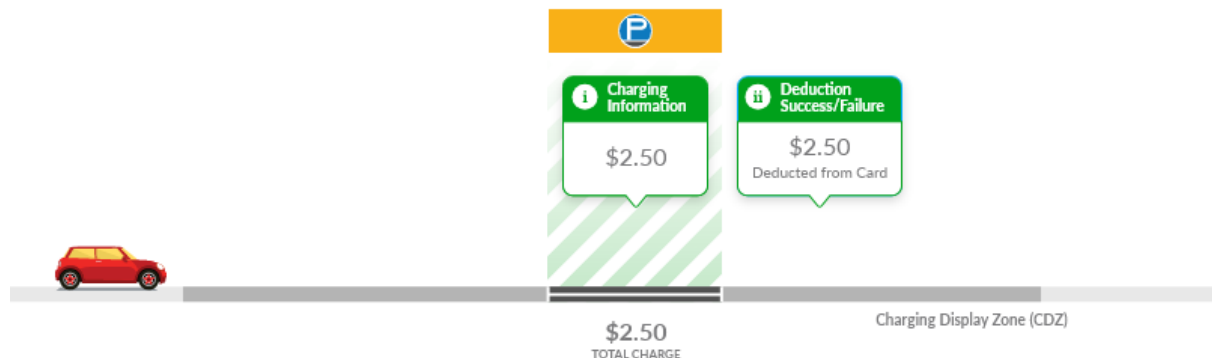


Figure 1-4 Enhanced Electronic Parking

- i. **Entry-Exit Charging Information:** This informs the motorist of the amount that will be deducted upon entering the parking area.
- ii. **Entry-Exit Deduction Successful/Failure:** This message displays the actual deducted amount at the exit point of a parking area when successful, and the expected deduction amount when unsuccessful.

1.6 Common Alert

This will display the common alerts available for the ERP 2.0. The message is used to send out road information, as shown in the image.

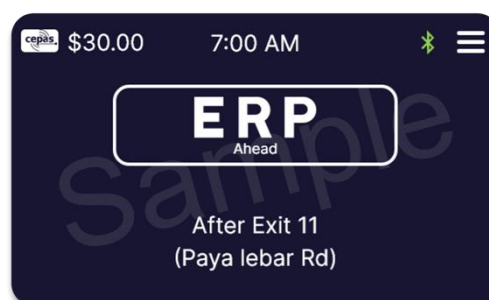


Figure 1-5 Common Alert Display

1.7 REP (Roadside Electronic Parking)

Roadside Electronic Parking is a new enhancement of the parking system for roadside parking to start a parking session for roadside parking and pay for the charges seamlessly.

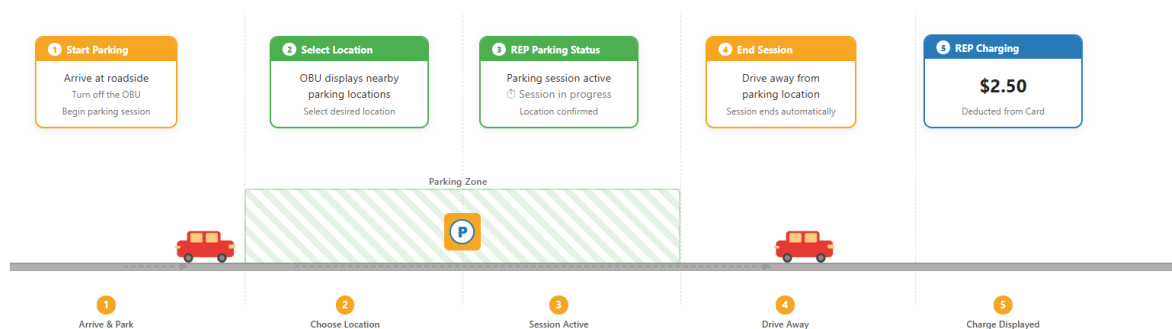


Figure 1-6 Roadside Electronic Parking

1.8 OPC (Off-Peak Car)

Driving an Off-Peak Car during restricted hours, motorist must buy an e-Day Licence for usage and this will be charged on the OBU when vehicle is travelling on public road.

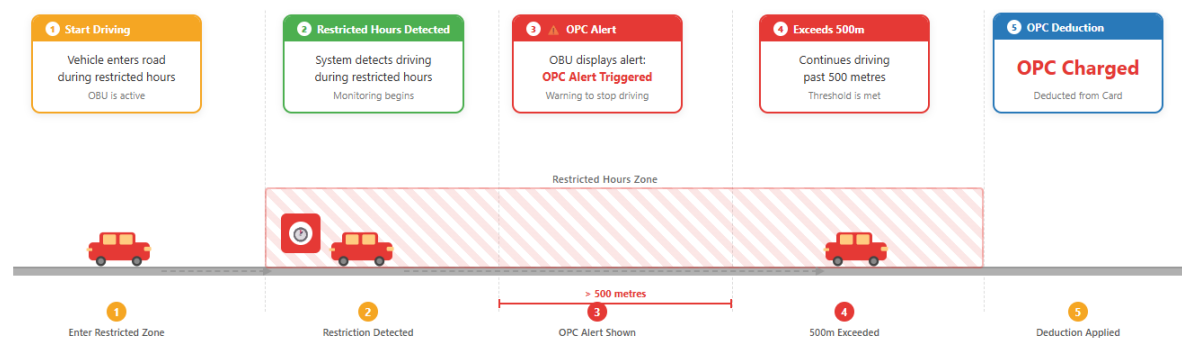


Figure 1-7 Off-Peak Car

1.9 CPT (Check Point Tolling)

Check Point Tolling charging takes place at a predefined point on the road and the OBU sends the following messages:

- i. **CPT Alert:** This message informs the motorist on the charges that will be incurred during a specific timeframe at the CDZ. It displays the road name, time range and road pricing charge.
- ii. **CPT Charging Information:** This message displays the amount that will be deducted at the CDZ.
- iii. **CPT Deduction Successful/Failure:** This message displays the actual deducted amount at the CDZ. The message contains information regarding successful and unsuccessful deductions.

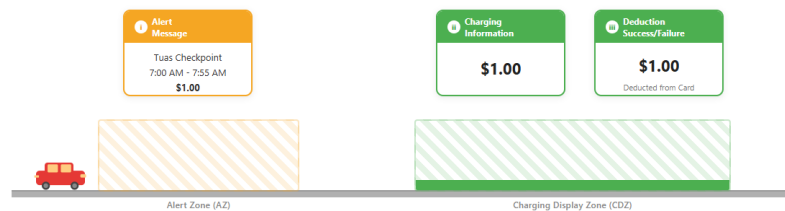


Figure 1-8 Check Point Tolling

1.10 Traffic Information (TDCID)

The SDK provides a set of Traffic related messages that are sent out from the OBU while on the road. These messages contain information about traffic conditions ahead of motorists and are divided into three categories:

i. General Traffic Information



Figure 1-9 General Traffic Information (School Zone)

This group contains messages related to traffic safety information, traffic jams ahead, speed limit information, school zones, silver zones and emergency messages. Typically, the data sent by the OBU includes an image that depicts the type of message, as well as a few rows of text that provide more information about the event.

ii. Travel Time Information

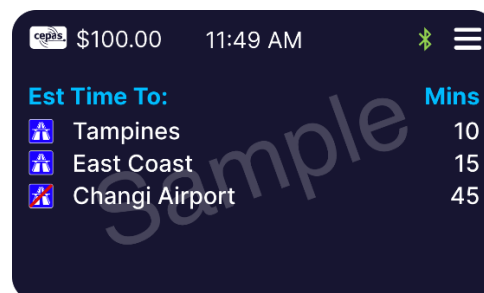


Figure 1-10 Travel Time Information

This message is sent to the motorist and provides an estimate of the travel time to popular locations based on their current location and heading.

iii. Parking Lot Information

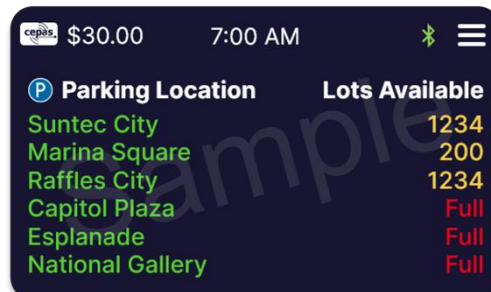


Figure 1-11 Parking Lot Information

This message informs the motorist of available parking lots near the motorist's current location.

2 GETTING STARTED

This documentation covers the following SDK version.

- iOS SDK version 3.0.0 and above
- Android SDK version 3.0.0 and above

For SDK version older than 3.0.0, please refer to the [developer guide version 1.5](#).

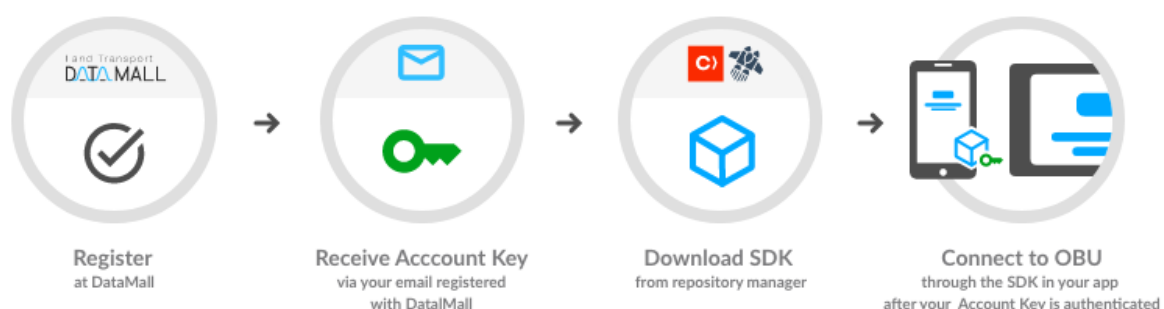


Figure 1-12 Developer Workflow Overview

These are the steps required for developers to get started with the SDK:

- Register at DataMall (<https://datamall.lta.gov.sg/content/datamall/en/request-for-api.html>).
- After registering, the developer will receive an email containing the SDK account key and a link to the developer guide.
- Follow the guide to integrate the SDK into your applications.

2.1 Getting SDK Account Key/Sign Up

To get started with the SDK, you will need an SDK Account key which can be obtained by registering at DataMall. After submitting the form, you will receive two emails:

- The first email will contain the DataMall API Access Key, which you need to access the DataMall APIs.
- The second email will contain the SDK Account Key for the SDK.

2.2 Sample Application with SDK Integration

Sample applications demonstrating the SDK integration are available on GitHub:

iOS: <https://github.com/extolta/obusdk-ios-sample>

Android: <https://github.com/extolta/obusdk-android-sample>

2.3 Register Your Phone at e-Services Portal

The motorist needs to register their iPhone or Android phone's Bluetooth MAC Address at e-Services portal. You can register up to a maximum of five devices per OBU. Once registered, the OBU will authenticate the Bluetooth MAC addresses of the paired phones against the registered list. Further details on Bluetooth MAC address authentication for both iOS and Android can be found below.

2.3.1 Bluetooth MAC Address Authentication for Android Phones

During Bluetooth pairing, Android OS automatically shares the phone's Bluetooth MAC address with the OBU. The OBU will then verify the Bluetooth MAC address against the list of registered addresses on the e-Services Portal for authentication. If authentication fails, an error will be displayed.

2.3.2 Bluetooth MAC Address Authentication for iPhones

When pairing an iPhone with the OBU, the device's Bluetooth MAC address must be manually inputted by the motorist through an on-screen prompt (Figure 1-15). The OBU will then verify the Bluetooth MAC address against the list of registered addresses on the e-Services Portal for authentication.

Please ensure that you enter the Bluetooth MAC address registered to the OBU on the screen below. The SDK will display this prompt whenever you connect to a new OBU.

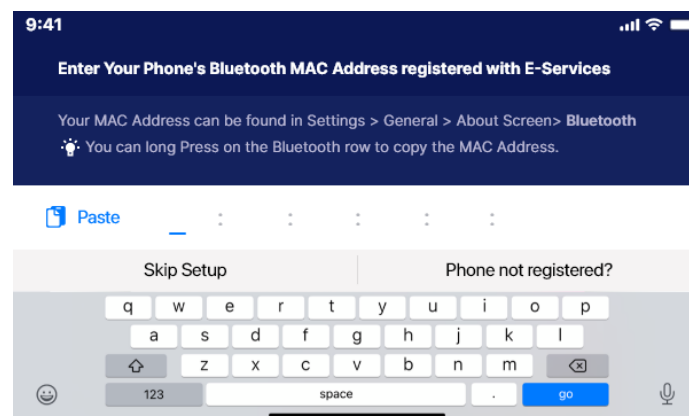


Figure 1-13 Bluetooth MAC Address Input Screen for iOS

3 ANDROID

3.1 Minimum API Level for the SDK

The SDK requires minimum API Level 26.

```
android {  
    defaultConfig {  
        minSdkVersion 26...  
    }  
}
```

3.2 Programming Language

The SDK has been developed in Kotlin. The recommended language of third-party applications is Kotlin. However, the SDK can also be used in Java applications.

3.3 Bluetooth Requirement

The communication between the OBU device and the SDK happens via Bluetooth. The OBU supports Bluetooth 1.0, 2.0+EDR, 4.0.

3.4 Third-Party Libraries

The SDK uses the following third-party libraries:

- i. Retrofit
- ii. SQLCipher
- iii. GSON

3.5 How to Add the SDK to Android Application

To include the SDK in your Android application, you can add it as a Maven dependency. Add the following to your top-level build.gradle file.

```
repositories {  
    google()  
    jcenter()  
    maven {  
        url 'https://extol.mycloudrepo.io/public/repositories/extol-android'  
    }  
}
```

Then, in your application's build.gradle file, add the following dependency:

```
dependencies {
    ....
    implementation 'sg.gov.lta:extol:<latest_version>'
    ...
}
```

Please refer to this [link](#) for the latest SDK version information.

3.6 SDK Permissions

The SDK requires the following permissions:

- i. Internet
- ii. Bluetooth permissions
- iii. OBU Data Access permission (OBU SDK custom permission)

3.6.1 SDK Custom Permission for Third-Party Applications

To protect the motorist's OBU data privacy, the SDK requires explicit consent in the form of a permission before collecting data from the OBU. The SDK requires applications to take the following custom permission from motorists:

```
<application's package name>.permission.OBU_DATA_ACCESS
```

For example, if your application's package is **com.example.myapp**, then the custom permission would be **com.example.myapp.permission.OBU_DATA_ACCESS**. This permission needs to be requested at runtime by the developer of an application using the SDK. Without granting this permission, the SDK methods will throw a run time exception.

The SDK requires the following permissions, which will be merged into your application's manifest after integrating the SDK. Therefore, you do not need to manually add them to your application's manifest:

```
<uses-permission android:name="android.permission.BLUETOOTH" /> <uses-permission
android:name="android.permission.BLUETOOTH_ADMIN" /> <uses-permission
android:name="android.permission.BLUETOOTH_CONNECT" /> <uses-permission
android:name="android.permission.BLUETOOTH_SCAN" /> <uses-permission
android:name="android.permission.INTERNET" /> <uses-permission
android:name="android.permission.ACCESS_NETWORK_STATE" />
```

3.7 Other Requirements

The SDK utilises the Companion Device API to scan for nearby OBUs. This API indicates to the system that your application intends to establish companion device connections.

```
<uses-feature android:name="android.software.companion_device_setup"
android:required="true" />
```

3.8 How to Integrate the SDK

3.8.1 Step 1: Initialise the SDK with an SDK Account Key

The first step is to initialise the SDK with your SDK Account Key, which is sent to you after registration at DataMall. The function takes an optional OBUInitListener to give you success and failure callbacks. The SDK will only work after it has been successfully initialised. The onError() method returns a list of error codes defined below:

```
val initListener = object : OBUInitListener {
    override fun onSuccess() {
        // Initialization was successful
    }

    override fun onError(error: OBUError) {
        // Initialization failed with error
    }
}

OBUSDK.init(cocontext, "<SDK_ACCOUNT_KEY>", initListener)
```

OBUInitListener:

1	fun onSuccess()
	This is invoked upon the successful authentication of the SDK Account key.
2	fun onError(error: OBUError)
	The possible errors are defined below: i. INVALID_SDK_ACCOUNT_KEY: if the SDK Account is not valid. ii. INTERNET_NOT_AVAILABLE: There is no internet connectivity on your phone. iii. DEVELOPER_UNAUTHORISED: The developer is not authorised to use the SDK. Please contact support if your SDK Account key has been blocked. iv. APPLICATION_UNAUTHORISED: your application with the SDK Account Key is not authorised. Please contact support if your application has been blocked.

v.	DEVELOPER_DEACTIVATED: Please contact support if you come across with error.
vi.	SERVER_ERROR: Server is not reachable. Please try again after some time. If the error persists, please contact support.
vii.	REQUEST_TIMEOUT: This happens if internet is slow, or server takes longer to reply.



Note: This method must be called before calling any other SDK APIs, otherwise a runtime exception will be thrown. It is recommended to call `OBUSDK.init()` from your Application class.

3.8.2 Step 2: OBU Data Permission

The SDK requires developers to obtain explicit OBU Data Access permission from the motorist prior to providing access to any OBU data. It is essential to verify that this permission has been granted before connecting to an OBU; failure to do so will result in an `OBUSDKDataAccessPermissionDenied` error callback being delivered to the registered listener, and the SDK will not proceed with the requested operation

Calling the following method shows the OBU Data permission alert to the motorist. The sample code is shown below.

```
requestPermissions(arrayOf("<application's package name>.permission.OBU_DATA_ACCESS"),
REQUEST_CODE)

override fun onRequestPermissionsResult(
    requestCode: Int,
    permissions: Array<out String>,
    grantResults: IntArray
) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults)
    if(requestCode == REQUEST_CODE && grantResults[0] ==
PackageManager.PERMISSION_GRANTED) {
        // startListening() or connect() can be called here
    }
}
```

3.8.3 Step 3: Establish a Connection to the OBU

To make a connection with the OBU, the first step is to search for OBUs.

3.8.3.1 Search for OBUs

Calling this function starts a search and scan for nearby OBUs. The SDK uses the companion device pairing API to accomplish this task. Once the SDK discovers OBUs, it presents a

system pop-up containing a list of OBUs for the motorist to choose from. The method takes an `OBUSearchListener` to receive callbacks regarding the search.

Note: Location services are required for searching OBUs.

```
OBUSDK.startSearch(listener: OBUSearchListener)
```

```
val searchListener = object : OBUSearchListener {
    override fun onObuSelected(obu: OBUSDK) {
    }
    override fun onPairing() {
    }

    override fun onSearchFailure(error: OBUSDK?) {
    }
}

OBUSDK.startSearch(searchListener)
```

1	<code>fun onObuSelected(obu: <code>OBUSDK</code>)</code>
	obu: <code>OBUSDK</code> After the motorist selects an OBU from the search dialogue, this method returns an <code>OBUSDK</code> object. The developer can then call the <code>connect()</code> method directly to establish a connection with the selected OBU.
2	<code>fun onPairing()</code>
	This function is invoked when the OBU being connected to has not been previously bonded with the device, indicating that the pairing process with commence.
3	<code>fun onSearchFailure(error: <code>OBUSDK</code>?)</code>
	This function is called. i. OBUSDKInternalSearchError: when an error occurs in the Companion Device API search. This can happen if devices are not found or if there is a search timeout. If such an error occurs, you are required to restart the search. Please ensure that Bluetooth is enabled on your phone before initiating the search again. ii. OBUSDKBluetoothdisabledError: When Bluetooth is disabled.

The system pop-up looks like the one shown below. The title on the dialogue '**Link with <App_Name>**' will be replaced by your application name. The list shows the available OBUs that have been discovered during the search. The user can select one of the OBUs to pair with the application.

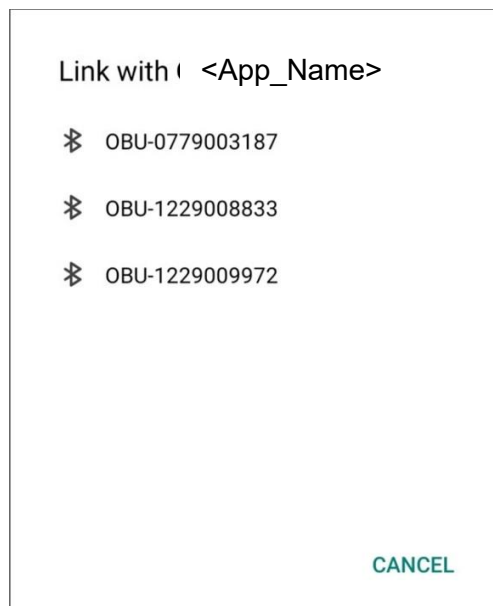


Figure 1-14 OBU Search Screen



Note: The OBU name format is in OBU-XXXXXXXXXX

3.8.3.2 Connect to the OBU

Once OBUs are discovered using the `startSearch()` method described above, the next step is to connect to one of the detected OBUs. This can be done by selecting an OBU from the list shown in the previous dialog. Connection status updates and callbacks are handled by `OBUConnectionStatusListener`, which is explained in the following section.

The SDK provides two `connect()` methods to establish a connection with the OBU:

- One method accepts an OBU object (typically received through the callbacks defined in `OBUConnectionStatusListener`)
- The other method accepts a vehicle number associated with a registered OBU

Details of both methods are provided below:

```
fun connect(obu: OBU, listener: OBUConnectionStatusListener)
```

This method allows a direct connection to an OBU discovered during the search phase.

```
fun connect(vehicleNumber: String, listener: OBUConnectionStatusListener)
```

This method allows the SDK to connect to a previously paired OBU using its associated vehicle number. The following conditions must be met for the connection to succeed:

- The OBU must already be paired with the smartphone via Bluetooth.
- Bluetooth must be enabled on the device.
- The OBU must be within Bluetooth range.

To disconnect the current connection to the OBU, use the following method.

```
fun disconnect()
```

3.8.3.3 OBUConnectionStatusListener

The connection listeners provide callbacks related to the connection. All methods in this listener are optional.

```
val connectionListener = object : OBUConnectionStatusListener {
    override fun onOBUConnected(device: OBU) {
    }
    override fun onOBUConnectionFailure(error: OBUError) {
    }
    override fun onOBUDisconnected(device: OBU?, error: OBUError?) {
    }
}
OBUSDK.connect(obu, connectionListener)
```

The description of each method is shown below:

1	fun onOBUConnected(device: OBU)
	This function is called when the connection is established with the OBU.
2	fun onOBUConnectionFailure(error: OBUError)
	error: This function is called when the SDK is unable to establish a connection with the OBU due to one of the possible error codes mentioned below: OBUInternalError: Please contact EXTOL customer support if this error persists. OBUNotPairedError: When connecting using vehicle number via fun connect(vehicleNumber, connectionlistener) defined in section 0 , this error is thrown if the OBU registered with the Vehicle number has not been paired with the phone. OBUSDKDataAccessPermissionDenied: In case the permission is not granted or revoked. The application must explicitly ask the OBU_DATA_ACCESS permission from the user and the user must “Allow” it.

	iv. OBUInternetNotAvailableError: Internet is required for some cases. This is returned if internet is not available on the phone (whether Wi-Fi or mobile data). v. OBUConnectionFailedError: This is thrown if an error occurs during establishing a Bluetooth connection with an OBU device. vi. OBUSDKPublicKeyForOBUNavailable(NO_PUBLIC_KEY_AVAILABLE): This is an internal error. If this occurs, make sure that your phone is registered with e-Services portal for your OBU.
3	fun onOBUDisconnected(device: OBU?, error: OBUError?) This function is called when the established connection with the OBU has been disconnected due to the possible errors that can be returned are as follows: i. OBUInternalError(SERVER_ERROR): Some unexpected value/ result was returned. ii. OBUSDKPublicKeyForOBUNavailable(NO_PUBLIC_KEY_AVAILABLE): This is an internal error. If this occurs, make sure that your phone is registered with e-Services portal for your OBU, iii. OBUSDKCodeSignatureFail(CODE_SIGNATURE_FAIL): The data signature validation failed. iv. OBUSDKDataAccessPermissionDenied(OBU_DATA_ACCESS_PERMISSION_DENIED): This error is returned if the user denies or revokes the SDK Data Access Permissions. v. OBUBadRequestError(BAD_REQUEST): Something went wrong with the server. Please contact support and try again after some time. vi. OBUInternetNotAvailableError(INTERNET_NOT_AVAILABLE): Internet is not available on the device.

3.8.3.4 Bluetooth Listener

This interface provides the change of states of the Bluetooth adapter.

```
private val bluetoothListener = object : OBUBluetoothListener {
    override fun onBluetoothStateUpdated(state: BluetoothState) {
        if (state == BluetoothState.STATE_ON) {
        }
    }
}

OBUSDK.setBluetoothListener(bluetoothListener)
```

	fun onBluetoothStateUpdated(state: BluetoothState)
1	These are the possible states: STATE_TURNING_OFF, STATE_TURNING_ON, STATE_OFF, STATE_ON

3.8.3.5 Subscribe to the OBUDataListener

Once the connection is established, the OBUDataListener is ready to receive data from the OBU. All functions in the interface have default (empty) implementations, so you only need to override the ones relevant to your application. Note that the “ChargingInformation” class has been replaced by the “ERPMessage” class in the SDK version 3.x. For full details on the new class structure, refer to [section 3.8.3.6](#).

```
Val dataListener = object : OBUDataListener {
    override fun onERPMessage(erpMessages: List<ERPMessage>) {
    }
    override fun onERPMessageAndTrafficInfo (
        trafficInfo: OBUTrafficInfo<out Any>?,
        erpMessages: List< ERPMessage >?
    ) {
    }
    override fun onVelocityInformation(velocity: Double) {
    }
    override fun onAccelerationInformation(acceleration: OBUAcceleration) {
    }
    override fun onCardInformation(
```

```

        status: OBUCardStatus?,
        paymentMode: OBUPaymentMode?,
        chargingPaymentMode: OBUCargingPaymentMode?,
        balance: Int?
    ) {
    }

    override fun onTravelSummary(travelSummary: OBUTravelSummary) {
    }

    override fun onPaymentHistories(histories: List<OBUPaymentHistory>) {
    }

    override fun onTrafficInformation(trafficInfo: OBUTrafficInfo<out Any>) {
    }

    override fun onError(error: OBUError) {
    }

    override fun onTotalTripCharged(totalCharged: OBUTotalTripCharged) {
    }
}

OBUSDK.setDataListener(dataListener)

```

1	<pre>fun onERPMessage(erpMessages: List<ERPMessage>)</pre> This function provides the list of ERP messages received from the OBU. ERP messages contain 12 type-safe subtypes representing charging events, alerts and parking information. See section 3.8.3.6 for the full list of message types. The list may contain multiple messages in the following common combinations: i. ChargingMessage + DeductionSuccessfulMessage ii. ChargingMessage + DeductionFailureMessage
2	<pre>fun onERPMessageAndTrafficInfo(trafficInfo: OBUTrafficInfo<out Any>?, erpMessage: List<ERPMessage>?)</pre> This function provides a unified callback for receiving ERP business function events and traffic information from the connected OBU in a type-safe, structured format. Both parameters are nullable. This callback is invoked when an ERP

	message or traffic information is received. If only one of them is received, the other parameter will be null. The details of using ERP are mentioned in section 3.8.3.6
3	<pre>fun onVelocityInformation(velocity: Double)</pre> This method provides velocity of the vehicle every second. The unit is m/s.
4	<pre>fun onAccelerationInformation(acceleration: OBUAcceleration)</pre> This method provides acceleration of the vehicle in every second. The unit is m/s².
5	<pre>fun onCardInformation(status: OBUCardStatus?, paymentMode: OBUPaymentMode?, chargingPaymentMode: OBUChargingPaymentMode?, balance: Int?)</pre> This method provides the Card Status, Payment Mode and Charging Payment Mode, and balance of the CEPAS card. OBUCardStatus: This represents the status for the CEPAS card inserted in the OBU. OBUPaymentMode: This represents the default payment mode of the OBU. If CEPAS card is inserted, it returns frontend, else backend OBUChargingPaymentMode: This represents the payment method used for the last deduction. This differs from the default payment mode only when the default method fails and the OBU falls back to an alternative (e.g., backend payment).
6	<pre>fun onTravelSummary(travelSummary: OBUTravelSummary)</pre> This method provides the total travel summary of the last trip. The SDK sends this callback only once after establishing the connection because the travel summary value remains same, as it represents the last trip summary. It contains the following info: Total Travel Time: The OBU calculates the Total Travel Time from the vehicle engine ignition ON until the ignition OFF. The unit of the value is seconds (sec). Total Travel Distance: The OBU calculates the estimated Total Travel Distance from the vehicle engine ignition ON to ignition OFF. The unit of the value is 10 meters (m).

7	fun onPaymentHistories(histories: List<OBUPaymentHistory>)
	Payment History is recorded when charging takes place. It consists of a record of the most recent 5 deductions from the current trip. For example, in a scenario with 7 deductions, the payment history will contain the latest 5 deductions, spanning from 7th to the 3rd deduction. In the event no charging activity, all records are reset to zero. Note: A trip represents the duration from OBU turning ON to OFF (car ignition on to OFF). When a new trip starts (OBU is switched on), the 5 payment history objects are zero. The payment by car park ticket on EEP without going through OBU is not recorded.
8	fun onTrafficInformation(info: OBUTrafficInfo<out Any>)
	It sends the Traffic Info from the OBU.
9	fun onError(error: OBUError)
	error: the possible errors related to the data i. OBUDeviceDataError: Thrown when the raw data is malformed. ii. OBUSDKDataAccessPermissionDenied: Thrown when data callback experiences permission not available. This could be the case if the permission was invoked while running the application from the phone's settings while the connection is established.
10	fun onTotalTripCharged(totalCharged: OBUTotalTripCharged)
	Total charge amount for ERP 2.0 from the ignition ON to OFF. Unit: cents. Note: The OBU calculates the total charged ERP 2.0 and EEP amounts for the trip from the vehicle engine ignition ON to ignition OFF. ERP1 and EPS charged amounts are not included in Total ERP 2.0 Charged amount.

3.8.3.6 ERPMessage Callback

ERPMessage is a type-safe sealed class representing 12 different message types for charging, parking, and alert events. Each message type is defined as a data class containing only the fields relevant to that specific event.

```
sealed class ERPMessage {
    abstract val businessFunction: OBUBusinessFunction
    data class AlertPointMessage(...) : ERPMessage()
    data class ChargingMessage(...) : ERPMessage()
    data class DeductionSuccessfulMessage(...) : ERPMessage()
```

```
data class DeductionFailureMessage(...) : ERPMessage()
data class MigrationMessage(...) : ERPMessage()
data class CarParkMessage(...) : ERPMessage()
data class SchemeUpdateMessage(...) : ERPMessage()
data class CommonAlertMessage(...) : ERPMessage()
data class ParkingStatusMessage(...) : ERPMessage()
}
```

3.8.3.6.1 Supported Message Types

Message Type	Business Function	Description
AlertPointMessage	ERP (Point Based), OPC, CPT	Advance alert before entering a charging zone.
ChargingMessage	ERP (Point Based or ERP1), EEP, EPS, OPC, REP, CPT	A pre-charging message sent before the actual deduction.
DeductionSuccessfulMessage	ERP (Point Based or ERP1), EEP, EPS, OPC, REP, CPT	Message indicating a successful deduction.
DeductionFailureMessage	ERP (Point Based or ERP1), EEP, EPS, OPC, REP, CPT	Message indicating a deduction failure and reason for the failure.
MigrationMessage	OPC, CPT	A message notifying motorist that CPT or OPC will begin soon.
CarParkMessage	EEP	A message from the EEP car park when the vehicle exits the gantry.
SchemeUpdateMessage	OPC	A message informing motorist that the vehicle is now regarded as an OPC vehicle (or non-OPC vehicle).
CommonAlertMessage	ERP	A general-purpose alert message not tied to any specific charging or parking event. Typically used to display contextual information such as road names. Contains up to two content fields.
ParkingStatusMessage	REP	Sent when a parking session begins. Provides the car park location, session start time, and the current parking fee.

3.8.3.6.2 Implementing ERPMessage Callback

Each message is associated with a specific business function. Depending on the use case, messages can be handled using one of the following approaches.

```

override fun onERPMessageAndTrafficInfo(
    trafficInfo: OBUTrafficInfo<out Any>?,
    erpMessage: List<ERPMessage>?
) {
    erpMessage?.forEach { message ->
        when (message) {
            is ERPMessage.AlertPointMessage -> {
                // Approach 1: Check business function and handle separately
                when (message.businessFunction) {
                    OBUBusinessFunction.ERP -> handleERPAlert(message)
                    OBUBusinessFunction.OPC -> handleOPCAAlert(message)
                    else -> {}
                }
            }

            is ERPMessage.DeductionSuccessfulMessage -> {
                // Approach 2: Handle regardless of business function
                handleDeductionSuccess(
                    amount = message.chargingAmount,
                    paymentMode = message.paymentMode
                )
            }

            // Handle other message types as needed
            else -> {}
        }
    }
}

```

3.8.3.7 ERPMessage Types

3.8.3.7.1 AlertPointMessage

Alert point message provides advance notification to drivers before entering a charging zone.

```

data class AlertPointMessage(
    override val businessFunction: OBUBusinessFunction,
    val chargingType: OBUCargingType?,
    val chargingAmount: Int,
    val content1: String,
    val content2: String?,
    val startTime: String,
    val endTime: String
) : ERPMessage()

```

Field	Type	Description
businessFunction	OBUBusinessFunction	ERP, OPC, CPT
chargingType	OBUChargingType?	To identify the ERP type (Point-Based)
chargingAmount	Int	Charging amount in cents
content1	String	Location or road name
content2	String?	Optional secondary message content such as a location name or additional details.
startTime	String	Start time of the road pricing. Format: yyyyMMddHHmmss
endTime	String	End time of the road pricing. Format: yyyyMMddHHmmss

3.8.3.7.2 ChargingMessage

Charging notification is sent before the actual deduction occurs. This message indicates that a charge is about to be applied and is always followed by either a DeductionSuccessfulMessage or DeductionFailureMessage.

```
data class ChargingMessage(
    override val businessFunction: OBUBusinessFunction,
    val chargingType: OBUChargingType?,
    val chargingAmount: Int
) : ERPMessage()
```

Field	Type	Description
businessFunction	OBUBusinessFunction	One of the following: ERP, EPS, EEP, OPC, CPT, REP
chargingType	OBUChargingType?	To identify the ERP type (Point-Based or ERP1)
chargingAmount	Int	Charging amount in cents

3.8.3.7.3 DeductionSuccessfulMessage

DeductionSuccessfulMessage confirms that a payment deduction was successfully processed. This message includes the deducted amount and the payment mode used (Frontend or Backend).

```
data class DeductionSuccessfulMessage(
    override val businessFunction: OBUBusinessFunction,
```

```
val chargingType: OBUChargingType?,
val chargingAmount: Int,
val paymentMode: OBUChargingPaymentMode
) : ERPMessage(
```

Field	Type	Description
businessFunction	OBUBusinessFunction	One of the following: ERP, EPS, EEP OPC, CPT, REP
chargingType	OBUChargingType?	To identify the ERP type (Point-Based or ERP1)
chargingAmount	Int	Charging amount in cents
paymentMode	OBUChargingPaymentMode	The payment method used for the deduction either frontend (CEPAS card) or backend.

3.8.3.7.4 DeductionFailureMessage

DeductionFailureMessage indicates that a payment deduction has failed. This message includes the card status, a descriptive failure reason, and the payment mode used for the deduction.

```
data class DeductionFailureMessage(
    override val businessFunction: OBUBusinessFunction,
    val chargingType: OBUChargingType?,
    val chargingAmount: Int,
    val paymentMode: OBUChargingPaymentMode,
    val cardStatus: OBUCardStatus,
    val failureMessage: String
) : ERPMessage()
```

Field	Type	Description
businessFunction	OBUBusinessFunction	One of the following: ERP, EPS, EEP, OPC, CPT, REP
chargingType	OBUChargingType?	To identify the ERP type (Point-Based or ERP1)
chargingAmount	Int	Charging amount in cents

paymentMode	OBUCHargingPaymentMode	The payment method used for the deduction either frontend (CEPAS card) or backend.
cardStatus	OBUCardStatus	The card status indicating the reason for the deduction failure (e.g., insufficientStoredValue, blacklisted, expiredStoredValue). Refer to section 6.8.1 for the complete list of card statuses.
failureMessage	String	Description of the failure reason

3.8.3.7.5 MigrationMessage

MigrationMessage notifies motorists that the commencement of CPT or OPC is approaching.

```
data class MigrationMessage(
    override val businessFunction: OBUBusinessFunction,
    val migrationMessage: String
) : ERPMessage()
```

Field	Type	Description
businessFunction	OBUBusinessFunction	OPC, CPT
migrationMessage	String	Migration message

3.8.3.7.6 CarParkMessage

CarParkMessage is sent from the EEP car park when the vehicle exits the gantry. The isFromParkingOwner flag indicates whether the message originates from the car park owner or the car park system.

```
data class CarParkMessage(
    override val businessFunction: OBUBusinessFunction,
    val content1: String?,
    val content2: String?,
    val content3: String?,
    val content4: String?,
    val content5: String?,
    val isFromParkingOwner: Boolean
) : ERPMessage()
```

Field	Type	Description
businessFunction	OBUBusinessFunction	EEP
content1	String?	Message from the carpark
content2	String?	Message from the carpark
Content3	String?	Message from the carpark
Content4	String?	Message from the carpark
Content5	String?	Message from the carpark
isFromParkingOwner	Boolean	If true, the message originates from the car park owner; otherwise, from the car park system.

3.8.3.7.7 SchemeUpdateMessage

SchemeUpdateMessage notifies the motorist that the vehicle is regarded as OPC vehicle (or non-OPC vehicle) from now on.

```
data class SchemeUpdateMessage(
    override val businessFunction: OBUBusinessFunction,
    val content1: String,
    val content2: String
) : ERPMessage()
```

Field	Type	Description
businessFunction	OBUBusinessFunction	OPC
content1	String	The scheme update message

content2	String	The name of the updated scheme
----------	--------	--------------------------------

3.8.3.7.8 CommonAlertMessage

CommonAlertMessage is a general alert message not specific to any charging or parking event. Contains up to two content fields for displaying informational messages to the driver.

```
data class CommonAlertMessage(
    override val businessFunction: OBUBusinessFunction,
    val content1: String?,
    val content2: String?
) : ERPMessage()
```

Field	Type	Description
businessFunction	OBUBusinessFunction	ERP
content1	String?	First line of alert message (e.g., "After Exit 11")
content2	String?	Second line of alert message (e.g., "Paya Lebar Rd")

3.8.3.7.9 ParkingStatusMessage

ParkingStatusMessage provides the current parking session status, including the car park location, session start time, and the current parking fee.

```
data class ParkingStatusMessage(
    override val businessFunction: OBUBusinessFunction,
    val carParkId: String,
    val carParkName: String,
    val startTime: String,
    val parkingFee: Int
) : ERPMessage()
```

Field	Type	Description
businessFunction	OBUBusinessFunction	REP
carParkId	String	Car park identifier
carParkName	String	Car park name

startTime	String	Parking session start time
parkingFee	Int	Current parking fee in cents

3.8.3.8 Migration Guide from OBUChargingInformation to ERPMessage

The SDK now uses a type-safe ERPMessage sealed class in place of the generic OBUChargingInformation and OBUChargingMessageType APIs. Each message type is a distinct subclass with its own strongly-typed fields, enabling compile-time safety and clearer code. This release also introduces five new message types such as MigrationMessage, SchemeUpdateMessage and ParkingStatusMessage, bringing the total to 12 messages.

Step 1: Update the import

Remove:

```
import sg.gov.lta.obu.sdk.core.models.OBUChargingInformation
import sg.gov.lta.obu.sdk.core.enums.OBUChargingMessageType
```

Add:

```
import sg.gov.lta.obu.sdk.core.models.ERPMessage
```

Step 2: Update OBUDataListener Callbacks

The following OBUDataListener callbacks have been renamed to better reflect the updated ERP messaging model.

onChargingInformation → onERPMessage

Before

```
override fun onChargingInformation(chargingInfo: OBUChargingInformation?) {
    // ...
}
```

After

```
override fun onERPMessage(erpMessages: ERPMessage?) {
    // ...
}
```

onErpChargingAndTrafficInfo → onERPMessageAndTrafficInfo

Before

```
override fun onErpChargingAndTrafficInfo(
    trafficInfo: OBUTrafficInfo<out Any>?,
    chargingInfo: List<OBUChargingInformation>?
)
```

After

```
override fun onERPMessageAndTrafficInfo(
    trafficInfo: OBUTrafficInfo<out Any>?,
    erpMessages: List<ERPMessage>?
)
```

Step 3: Migrate from OBUChargingInformation to ERPMessage

Previously, message handling required checking the `chargingMessageType` enum on a generic `OBUSChargingInformation` object. Now, `ERPMessage` is a sealed class with 12 specific subtypes. Use Kotlin's 'is' type checks in a 'when' expression to match the message and access its properties directly via smart cast.

Before

```
when (chargingInfo.messageType) {
    OBUSChargingMessageType.ALERT_POINT -> {
        val amount = chargingInfo.chargingAmount
        val content = chargingInfo.content1
        ...
    }
    OBUSChargingMessageType.CHARGING -> { ... }
    OBUSChargingMessageType.DEDUCTION_SUCCESS -> { ... }
}
```

After

```
when (erpMessage) {
    is ERPMessage.AlertPointMessage -> {
        val amount = erpMessage.chargingAmount
        val content = erpMessage.content1
        ...
    }
    is ERPMessage.ChargingMessage -> { ... }
    is ERPMessage.DeductionSuccessfulMessage -> { ... }
}
```

Type Mapping Reference

Old (OBUSChargingMessageType)	New (ERPMessage Subtype)
AlertPoint	ERPMessage.AlertPointMessage
Charging	ERPMessage.ChargingMessage
DeductionSuccessful	ERPMessage.DeductionSuccessfulMessage
DeductionFailure	ERPMessage.DeductionFailureMessage
CPOInformation	ERPMessage.CarParkMessage
DetectAlertPointCommonAlert	ERPMessage.CommonAlertMessage

Note: Since Common Alert is now a distinct message, all references `OBUSChargingType.Common` should be replaced with `ERPMessage.CommonAlertMessage`.

3.8.3.9 Code Migration Examples

3.8.3.9.1 Example 1: Alert Point Messages

Before: using `OBUSChargingInformation`

```
fun handleAlertPoint(data: OBUSChargingInformation) {
    val location = data.content1 // String?
    val amount = data.chargingAmount
    val chargingType = data.chargingType
    val dollars = amount / 100.0

    println("Approaching $location - $$dollars")

    // Check charging type
    when (chargingType) {
        OBUSChargingType.PointBased -> println("Point-based charging")
        else -> {}
    }
}
```

After: using `ERPMessage.AlertPointMessage`

```
fun handleAlertPoint(data: ERPMessage.AlertPointMessage) {
    val location = data.content1
    val amount = data.chargingAmount
    val chargingType = data.chargingType
    val dollars = amount / 100.0

    println("Approaching $location - $$dollars")
    println("Active: ${data.startTime} - ${data.endTime}")

    // Check charging type
    when (chargingType) {
        OBUSChargingType.PointBased -> println("Point-based charging")
    }
    else -> {}

    // Display alert to user
    showAlert(
        title = "ERP Alert",
        message = "Approaching $location\nCharge: ${String.format("%.2f", dollars)}"
    )
}
```

3.8.3.9.2 Example 2: Deduction Success Messages

Before: using `OBUSChargingInformation`

```
fun handleDeductionSuccess(data: OBUSChargingInformation) {
    val amount = data.chargingAmount
    val paymentMode = data.paymentMode
    val dollars = amount / 100.0
```

```
val mode = if (paymentMode == OBUChargingPaymentMode.Frontend) "Card" else "Account"

println("Payment successful: $$dollars")
println("Payment method: $mode")
}
```

After: using `ERPMessage.DeductionSuccessfulMessage`

```
fun handleDeductionSuccess(data: ERPMessage.DeductionSuccessfulMessage) {
    val amount = data.chargingAmount
    val paymentMode = data.paymentMode
    val dollars = amount / 100.0

    val mode = if (paymentMode == OBUChargingPaymentMode.Frontend) "Card" else "Account"

    println("Payment successful: $$dollars")
    println("Payment method: $mode")
}
```

3.8.4 SDK Helper Methods for OBU Data

1	<code>fun getCardBalance() : Int?</code>
	This function returns the last card balance if available.
2	<code>fun getPaymentMode() : OBUPaymentMode?</code>
	This function returns the OBU Payment Mode if available.
3	<code>fun getCardStatus() : OBUCardStatus?</code>
	This function returns the card status if available.
4	<code>fun getPaymentHistories() : List<OBUPaymentHistory>?</code>
	This function returns the list of payment transactions if available.
5	<code>fun getLastObuData() : OBUData?</code>
	This function returns the last received OBU data.
6	<code>fun getPairedDevices(): List<OBU>?</code>
	This function returns a list of OBUs that have been previously paired with the app.

7	<code>fun getLastTravelSummary() : OBUTravelSummary?</code>
	This function returns the last received Travel Summary data.
8	<code>fun getLastTrafficInfo() : OBUTrafficInfo?</code>
	This function returns the last received Traffic info.
9	<code>fun isConnectionActive() : Boolean</code>
	Returns the connection status.

3.8.5 Mock Manager for Android Applications

3.8.5.1 Overview

The OBU SDK includes a built-in Mock Manager that allows developers to simulate ERP charging messages and traffic information (TDCID) events without connecting to a physical OBU device. This is useful for development, testing, and demonstration purposes.

The Mock Manager uses three key classes:

- **MockedConnectionHandler:** configures the time interval, card balance, event sequence and cyclic mode
- **MockEvent:** a container that holds one or more ERPEvent messages and an optional TdcidEvent message
- **ERPEvent / TdcidEvent:** individual mock event messages for ERP and traffic information respectively

3.8.5.2 MockedConnectionHandler

MockedConnectionHandler configures the behaviour of the mock OBU connection.

Method	Type	Default	Description
setTimeInterval(Long)	Long	5	Time interval (in seconds) between each mock event in the sequence
setCardBalance(Int)	Int	500	Simulated card balance (in cents) reflected in mock messages
setSequence(List<MockEvent>)	List<MockEvent>	Default mock sequence	Ordered list of mock events
setCyclicMode(Boolean)	Boolean	false	When enabled, the sequence repeats continuously from the beginning after the last event

3.8.5.3 MockEvent

MockEvent is a container that represents a single step in the mock sequence. Each MockEvent can contain one or more ERP messages and an optional traffic information event, which are delivered together at each time interval.

MockEvent only requires ERP messages and/or TDCID events to be configured. All other data callbacks (velocity, acceleration, card information, etc.) are automatically generated by the mock and cannot be set programmatically.

Property	Type	Description
erpEvents	List<ERPEvent>?	One or more ERP mock events to be delivered
tdcidEvent	TdcidEvent?	An optional traffic information event to be delivered alongside the ERP events

You can create a MockEvent with both ERP and TDCID events, or with just one of them:

```
// ERP + TDCID
MockEvent(
    erpEvents = listOf(AlertPointMessage.Mock.erpPointBased()),
```

```

        tdcidEvent = TdcidEvent.Template1A()
    )

    // ERP only
    MockEvent(
        erpEvents = listOf(ChargingMessage.Mock.erpPointBased()),
        tdcidEvent = null
    )

    // TDCID only
    MockEvent(
        erpEvents = null,
        tdcidEvent = TdcidEvent.Template2B()
    )

```

3.8.5.4 ERPEvent

Each ERPMessage type has a corresponding ERPEvent sealed class with a nested Mock object that provides factory methods for creating pre-configured mock events. All parameters have default values and can be configured as needed (e.g. chargingAmount, content1, paymentMode, cardStatus).

Message Type	Mock Methods	Params (Optional)
AlertPointMessage	erpPointBased, opcSingleArea, opcMultiArea, cpt	chargingAmount: Int, content1: String *content2: String (only for opcMultiArea)
ChargingMessage	erpPointBased, erp1, eep, eps, opc, cpt, rep	chargingAmount: Int
DeductionSuccessfulMessage	erpPointBased, erp1, eep, eps, opc, cpt, rep	chargingAmount: Int, paymentMode: OBUChargingPaymentMode
DeductionFailureMessage	erpPointBased, erp1, eep, eps, opc, cpt, rep	chargingAmount: Int, paymentMode: OBUChargingPaymentMode
CommonAlertMessage	erp	content1: String, content2: String
CarParkMessage	ownerMessage, eepInformation	content1: String, content2: String, content3: String, content4: String, *content5: String (only for eepInformation)

SchemeUpdateMessage	opc	content1: String, content2: String
ParkingStatusMessage	parkingStatus,	carParkId: String, carParkName: String, parkingFee: Int
MigrationMessage	opc, cpt	migrationMessage: String

Example:

```
ERPEvent.AlertPointMessage.Mock.erpPointBased(chargingAmount = 200, content1 = "Ophir Road")

ERPEvent.DeductionSuccessfulMessage.Mock.erpPointBased(chargingAmount = 100, paymentMode = OBUChargingPaymentMode.Frontend)
```

3.8.5.5 TdcidEvent

TdcidEvent is a sealed class that provides pre-defined traffic information display templates. Each subclass represents a specific TDCID screen layout.

Template Range	Screen ID	Description
template1A – template1V	Screen 1	For unplanned event
template2A – template2E	Screen 2	
template3A – template3C	Screen 3	For free text layout
template4A – template4B	Screen 4	
template5A – template5B	Screen 5	
template6A – template6B	Screen 6	
template8A – template8Q	Screen 8	
template110A – template110B	Screen 110	For parking guidance system
template111A – template111B	Screen 111	For travel time display
template12A – template12R	Screen 12	For preloaded message
template13A	Screen 13	For speed limit

3.8.5.6 Setting Up Mock Manager

```
val mockManager = MockedConnectionHandler.Builder()
    .setCardBalance(10000) // $100.00
    .setCyclicMode(false)
    .setSequence(listOf(
        // alert
```

```
MockEvent(
    listOf(ERPEvent.AlertPointMessage.Mock.erpPointBased(200)),
    TdcidEvent.Template1B()
),
//Charging
MockEvent(
    listOf(ERPEvent.ChargingMessage.Mock.erpPointBased(200)),
    null
),
// Deduction successful
MockEvent(
    listOf(ERPEvent.DeductionSuccessfulMessage.Mock.erpPointBased(200)),
    TdcidEvent.Template110A()
)
))
.build()

// Enable mock data before connecting
OBUSDK.enableMockData(mockManager)
```

3.8.5.7 Simulating Concurrent Messages

When charging segments are close together (e.g. charging and deduction) on the road, the OBU may deliver multiple messages at the same time. To simulate this, add multiple ERPEvents to a single MockEvent.

```
MockEvent(
    listOf(
        ERPEvent.ChargingMessage.Mock.erpPointBased(chargingAmount = 50),
        ERPEvent.DeductionSuccessfulMessage.Mock.erpPointBased(chargingAmount = 50)
    ),
    TdcidEvent.Template110A()
)
```

3.8.5.8 How to Use Mock Manager

Step 1: Initialize the SDK

Call `OBUSDK.init(context, sdkKey, initListener)` to initialize the SDK with API key.

Step 2: Enable Mock Data

Call `OBUSDK.enableMockData(mockedConnectionHandler)` to enable a mock environment for OBU connection. This must be called before initiating a connection.

Step 3: Start Search

Call `OBUSDK.startSearch(searchListener)` to begin searching for a mock device. A simulated OBU is returned via the `onOBUSelected(obu: OBU)` callback on the search listener.

Step 4: Subscribe data listener

Call `OBUSDK.setDataListener(dataListener)` to receive data callbacks e.g. `onERPMessageAndTrafficInfo`, `onTravelSummary`, `onPaymentHistories`.

Step 5: Connect to Mock

Call `OBUSDK.connect(obu, connectionStatusListener)` with the mock OBU received in Step 3 to establish the connection.

Step 6: Disconnect

Call `OBUSDK.disconnect()` to disconnect from the mock OBU. In non-cyclic mode, `onOBUDisconnected` is called with a `MockSequenceCompletedException` when the configured sequence has completed.

3.8.6 Traffic Icons in the OBU SDK

The SDK includes a bundle of icons specifically designed for Traffic messages, which can be utilised by applications. Each `TrafficInfo` message is associated with an icon that corresponds to a unique icon ID within the SDK. Refer to Appendix A for traffic icons list.

The field `icon` in `OBUTrafficInfo` (section Traffic Information) contains the id of the image. The following function in the SDK can be used to retrieve the resource ID:

```
fun getTDCIDImageRes(id: String): Int // return the resource id
```

After getting the resource ID, the `getDrawables` can be used to obtain the image as shown below:

```
resources.getDrawable(getTDCIDImageRes(icon), null) // get icon from OBUTrafficInfo
```

4. IOS

4.1 Minimum iOS Version

The SDK supports iOS 13 and higher. Third-party applications are required to set the iOS minimum deployment target to iOS 13.

4.2 Programming Language

The SDK is developed in Swift, which is the recommended language for third-party applications. Objective-C is also supported.

4.3 Bluetooth Requirement

The OBU communicates with the SDK via Bluetooth BT 4.1 + LE.

4.4 How to Add the SDK to iOS Applications

The SDK can be installed or updated either CocoaPods or Swift Package Manager (SPM). New projects should use Swift Package Manager as **CocoaPods will reach end-of-life in December 2026**.

4.4.1 CocoaPods

CocoaPods can be installed from the terminal by running the following command:

```
sudo gem install cocoapods
```

After installing it, run the following:

```
pod init
```

Add the following Pod into the podfile and run the command 'pod install':

```
pod "extol"
```

Refer to this [link](#) for the latest SDK version information.

⚠ Note: As CocoaPods will reach end-of-life in December 2026, EXTOL SDK will not publishing any updates to CocoaPods from December 2026 onwards. Developers are strongly recommended to migrate the EXTOL SDK integration from CocoaPods to Swift PM to ensure continued access to the latest updates and support.

To have a seamless migration of existing device-related data, Developers should upgrade from the current CocoaPods (with RealmSwift) to the enhanced CocoaPods (with Encrypted File Storage and RealmSwift). This intermediate upgrade will ensure that device-related data is migrated to the new SDK architecture before proceeding with SwiftPM migration.

4.4.2 Swift Package Manager (SPM)

The SDK is also available as an XCFramework distributed via Swift Package Manager.

Below are the steps for accessing the SDK using SPM:

1. In Xcode, go to File → Add Package Dependencies
2. Enter the SDK repository URL: <https://github.com/extolta/obusdk-ios-spm.git>
3. Select the version rule as Exact Version (recommended): e.g.: 1.0.0
4. Click Add Package
5. Select the OBUSDK library and add it to your app target
6. After adding the package, import the SDK in your Swift files:

```
import OBUSDK
```

Refer to this [link](#) for the latest SDK version information.

4.5 SDK Permissions

The SDK communicates with the OBU via Bluetooth and requires Bluetooth access permission. For that, a key `NSBluetoothAlwaysUsageDescription` needs to be added to the project's info.plist file. The description tells the user why the application needs access to Bluetooth. For instance:

“Bluetooth is required to receive ERP and Traffic messages from the OBU”

Your application will crash **if** the Info.plist does not have **this** key:
`NSBluetoothAlwaysUsageDescription`.

4.6 How to Integrate the SDK

The SDK provides `OBUSDKManager`, which is a singleton class, and acts as an entry point of the SDK to call all the methods. It can be accessed as:

`OBUSDKManager.shared`

The SDK for iOS provides a **Handler** that represents either **success** or **failure**, including an associated value in each case.

```
public typealias Handler<T> = (HandlerResult<T, NSError>) -> Void

public enum HandlerResult<Success, Failure> {
    case success(Success)
    case failure(Failure)
}
```

4.6.1 Step 1: Initialise SDK with the SDK Account Key

The first step is to initialise the SDK by entering the key sent to your email after registration at DataMall in the method below. It can be called from the AppDelegate of your application.

```
public func initializeSDK(with sdkKey: String, _ completion: @escaping Handler<Bool>)
// Replace the SDK Account key with your key.
OBUSDKManager.shared.initializeSDK(with: <SDK_ACCOUNT_KEY>) { result in
    switch result {
    case .success(let status):
        print("initialised")
    case .failure(let error):
```

```
        print("failed with error code: \(error.code)")
    }
} }
```

4.6.2 Step 2: OBU Data Permission

The SDK requires developers to obtain explicit OBU Data Access permission from the motorist prior to providing access to any OBU data. It is essential to verify that this permission has been granted before connecting to an OBU; failure to do so will result in an `OBUSDKDataAccessPermissionDenied` error callback being delivered to the registered listener, and the SDK will not proceed with the requested operation.

The following method is used to check the status of the permission.

```
// return the status of the permission granted or not
func isDataPermissionGranted() -> Bool
```

This method is used to present the permission alert dialogue to the motorist. If the permission is denied, this method can be called again to request permission. The handler parameter can be used to determine the status of the permission.

```
func requestOBUDataAccessPermission(_ completion: @escaping Handler<Bool>)
```

The sample code is shown below:

```
if ! OBUSDKManager.shared.isDataPermissionGranted() {
    OBUSDKManager.shared.requestOBUDataAccessPermission { result in
        switch result {
        case .success(let status):
            print("Permission granted")
        case .failure(let error):
            print("Permission not granted")
        }
    }
}
```

4.6.3 Step 3: Search for OBUs

The SDK provides a delegate to receive search and connection related callbacks. The application must conform to this delegate.

```
OBUSDKManager.shared.connectionStatusDelegate = self
```

This method initiates a search to find nearby OBUs and returns a list of nearby OBUs to the delegate method defined below. The delegate must be implemented before calling the search method.

```
OBUSDKManager.shared.startSearch()
```

This method is triggered when a new device is found during the scanning process, which runs for 60 seconds. The application can listen to the callback and show the list of OBUs to the user for selection.

```
func onOBUFound(_ device: [OBU])
```

4.6.4 Connect to an OBU

The method establishes a connection to the OBU, which can be retrieved from the delegate method mentioned above. Each OBU object contains a name field that can be used to identify it among others. The SDK provides two connect() methods to establish a connection with the OBU:

- One method accepts an OBU object
- The other method accepts a vehicle number associated with a registered OBU

Details of both methods are provided below:

```
OBUSDKManager.shared.connect(obu: obu)
```

This method allows a direct connection to an OBU discovered during the search phase.

```
func connect(vehicleNumber: String)

OBUSDKManager.shared.connect(vehicleNumber: "SME1111Z")
```

This method allows the SDK to connect to a previously paired OBU using its associated vehicle number. The following conditions must be met for the connection to succeed:

- The OBU must already be paired with the smartphone via Bluetooth.
- Bluetooth must be enabled on the device.
- The OBU must be within Bluetooth range.

To disconnect the current connection to the OBU, use the following method

```
OBUSDKManager.shared.disconnect()
```

4.6.5 Connection Status Delegate

This delegate provides an interface for listening to the search and connection updates. The methods are defined below.

4.6.5.1 OBUConnectionDelegate

1	func onOBUConnected(_device: OBU)
	This method gets called when the connection with the OBU has been established.
2	func onOBUDisconnected(_device: OBU, error: NSError?)
	This method gets called when the connection with the OBU has been disconnected. The disconnection could occur due to internal Bluetooth disruptions or the OBU going out of Bluetooth range. The error object contains the error from the core Bluetooth framework.
3	func onOBUFound(_ device: [OBU])
	The startSearch() method returns a list of nearby OBUs, and the device continues scanning until the search is stopped after 60 seconds.
4	func onOBUConnectionFailure(_ error: NSError)
	This function is called when the connection fails. It could fail due to the following reasons: i. INTERNET_NOT_AVAILABLE: An Internet connection is required by the SDK to fetch OBU related data from the server. In case if there is not internet connectivity, this error is returned. ii. CONNECTION_FAILED: when the connection between the application and OBU is not successful. It could be due to Bluetooth disruption or Bluetooth out of range. iii. MAC_ADDRESS_AUTHENTICATION_FAILED: OBU authenticate your application against the Bluetooth MAC Address of your iPhone. Please ensure your iPhone is registered at 'e-Services' portal. The same Bluetooth MAC Address registered with the OBU must be keyed in the application. The Bluetooth MAC Address authentication screen is presented from the SDK during the connection process. Once added, the SDK will not show it again. iv. BLUETOOTH_DISABLED: when the Bluetooth is disabled. v. NO_PUBLIC_KEY_AVAILABLE: This is an internal error. If this occurs, make sure that your phone is registered with e-Services portal for your OBU, vi. CODE_SIGNATURE_FAIL: This is an internal SDK error. If this error persists, please contact the EXTOL Support Team. vii. OBU_NOT_PAIRRED: When connecting using vehicle number with func connectWith(vehicleId: String), this error is thrown if the OBU registered with the Vehicle number has not been paired with the phone. viii. OBU_DATA_ACCESS_PERMISSION_DENIED: This is returned if the OBU data permission has not been taken from the motorist. Upon Receiving this, the developer needs to call the function requestOBUDataAccessPermission().

5	func onBluetoothStateUpdated(_ state: BluetoothState)
	This function is called when the CBCentralManager changes its state. The states are as follows: i. Unknown ii. Resetting iii. Unsupported iv. Unauthorized v. Powered Off vi. Powered On
6	func onSearchFinished()
	The search runs for 60 seconds. This is a helper function to get the update that the search has stopped. The application can use this info to show a message to the user.

Once the connection has been established, the following data delegates are used to receive data from the OBU.

4.6.5.2 OBUDataDelegate

Once the connection is established, the data listener begins receiving data from the OBU. All callback methods have default implementations. You only override methods that your application needs. Note that OBUChargingInformation has been replaced by ERPMessage in this version. See section [4.6.6](#) for details on the new class structure.

```
OBUSDKManager.shared.dataDelegate = self
```

1	func onVelocityInformation(_ velocity: Double)
	This function provides velocity info of the vehicle in meter/second.
2	func onAccelerationInformation(_ acceleration: OBUAcceleration)
	This function provides acceleration info of the vehicle.
3	func onERPMessage(_ erpMessages: [ERPMessage])
	This function provides the list of ERP messages received from the OBU. Each message is one of 12 type-safe subtypes representing charging events, alerts and parking information. See section 4.6.7 for the full list of message types. The list may contain multiple messages in the following common combinations: i. ChargingMessage + DeductionSuccessfulMessage

	ii. ChargingMessage + DeductionFailureMessage
4	<pre>func onTrafficInformation(_ trafficInfo: OBUTrafficData)</pre> This function provides traffic info from the OBU.
5	<pre>func onERPMessageAndTrafficInfo(trafficInfo: OBUTrafficInfo?, erpMessages: [ERPMessage]?)</pre> This function provides a unified callback for receiving ERP business function events and traffic information from the connected OBU in a type-safe, structured format. <u>Both</u> parameters are nullable. This callback is invoked when an ERP message or traffic information is received. If only one of them is received, the other parameter will be null. The details of using ERP are mentioned in section below.
6	<pre>func onPaymentHistories(_ histories: [OBUPaymentHistory])</pre> This function always returns the last five charging objects during current trip from the OBU for ERP. For example, in a scenario with 7 deductions, the payment history will encompass the latest 5 deductions, spanning from 7th to the 3rd deduction. In case of less than five charging, the remaining objects will be empty but there will be always five objects. In the event that there is no charging activity, all records are set to zero.
7	<pre>func onTravelSummary(_ travelSummary: OBUTravelSummary)</pre> This method provides the total travel summary of the last trip. The SDK sends this callback only once after establishing the connection because the summary/value never changes, as it represents the last trip summary. It contains the following info: Total Travel Time: The OBU calculates the Total Travel Time from the vehicle engine ignition ON until the ignition OFF. The unit of the value is seconds (sec). Total Travel Distance: The OBU calculates the estimated Total Travel Distance from the vehicle engine ignition ON to ignition OFF. The unit of the value is 10 meters (m).
8	<pre>func onTotalTripCharged(_ totalCharged: OBUTotalTripCharged)</pre>

	Total charge amount for ERP 2.0 from the ignition ON to OFF. Unit: cents. Note: The OBU calculates the total charged ERP 2.0 and EEP amounts for the trip from the vehicle engine ignition ON to ignition OFF. ERP1 and EPS charged amounts are not included in Total ERP 2.0 Charged amount.
9	<pre>func onError(_ errorCode: NSError)</pre> This function provides the following error. i. DEVICE_DATA_ACCESS_ERROR: This is returned when the data is malformed. Please contact EXTOL Support Team if it persists.
10	<pre>func onCardInformation(_ status: OBUCardStatus?, paymentMode: OBUPaymentMode?, chargingPaymentMode: OBUCargingPaymentMode?, balance: Int)</pre> This function provides the following info of the inserted card: i. OBUCardStatus: This represents the status for the CEPAS card inserted in the OBU. ii. OBUPaymentMode: This represents the default payment mode of the OBU. If CEPAS card is inserted, it returns frontend, else backend iii. OBUCargingPaymentMode: This represents the payment method used for the last deduction. This differs from the default payment mode only when the default method fails and the OBU falls back to an alternative (e.g., backend payment).

4.6.6 ERPMessage Callback

ERPMessage is a type-safe sealed class representing 12 different message types for charging, parking, and alert events. Each message type is defined as a data class containing only the fields relevant to that specific event.

```
public enum ERPMessage: Equatable {
    case alertPoint(AlertPointMessage)
    case charging(ChargingMessage)
    case deductionSuccessful(DeductionSuccessfulMessage)
    case deductionFailure(DeductionFailureMessage)
    case migration(MigrationMessage)
    case carPark(CarParkMessage)
    case schemeUpdate(SchemeUpdateMessage)
    case commonAlert(CommonAlertMessage)
```

```

    case parkingStatus(ParkingStatusMessage)
}

```

4.6.6.1 Supported Message Types

Message Type	Business Function	Description
AlertPointMessage	ERP (Point Based), OPC, CPT	Advance alert before entering a charging zone
ChargingMessage	ERP (Point Based or ERP1), EEP, EPS, OPC, REP, CPT	A pre-charging message sent before the actual deduction
DeductionSuccessfulMessage	ERP (Point Based or ERP1), EEP, EPS, OPC, REP, CPT	Message indicating a successful deduction
DeductionFailureMessage	ERP (Point Based or ERP1), EEP, EPS, OPC, REP, CPT	Message indicating a deduction failure, and reason for the failure
MigrationMessage	OPC, CPT	A message notifying motorists that CPT or OPC will begin soon.
CarParkMessage	EEP	A message from the EEP car park when the vehicle exits the gantry.
SchemeUpdateMessage	OPC	A message informing motorists that the vehicle is now regarded as an OPC vehicle (or non-OPC vehicle).
CommonAlertMessage	ERP	A general-purpose alert message not tied to any specific charging or parking event. Typically used to display contextual information such as road names. Contains up to two content fields.
ParkingStatusMessage	REP	Sent when a parking session begins. Provides the car park location, session start time, and the current parking fee.

4.6.6.2 Implementing ERPMessage Callback

Each message is associated with a specific business function. Depending on the use case, messages can be handled using one of the following approaches.

```
func onERPMessageAndTrafficInfo(trafficInfo: OBUTrafficInfo?, erpMessages: [ERPMessage]?)
{
    guard let messages = messages else { return }

    for message in messages {
        switch message {
        case .alertPoint(let data):
            // AlertPointMessage is associated with different business
            // functions (e.g. ERP, OPC, CPT). The following demonstrates
            // per-function handling
            switch data.businessFunction {
            case .ERP:
                handleERPAlert(data)
            case .OPC:
                handleOPCAAlert(data)
            default:
                break
            }

        case .deductionSuccessful(let data):
            showSuccessNotification(
                amount: data.chargingAmount,
                paymentMode: data.paymentMode
            )

            // Handle other message types as needed
            default:
                break
        }
    }
}
```

4.6.7 ERPMessage Types

4.6.7.1 AlertPointMessage

Alert point message provides advance notification to drivers before entering a charging zone. These messages are typically received 200-300 meters before the charging point, giving drivers sufficient time to prepare for the charge or choose an alternative route.

```
public struct AlertPointMessage: Equatable {
    public let businessFunction: OBUBusinessFunction
    public let chargingType: OBUCargingType?
    public let chargingAmount: Int
    public let content1: String
    public let content2: String?
    public let startTime: String
    public let endTime: String
}
```

Field	Type	Description
businessFunction	OBUBusinessFunction	ERP, OPC, CPT
chargingType	OBUChargingType?	To identify the ERP type (Point-Based)
chargingAmount	Int	Charging amount in cents
content1	String	Location name or some other message
content2	String?	Optional secondary message content such as a location name or additional details.
startTime	String	Start time of the road pricing. Format: yyyyMMddHHmmss
endTime	String	End time of the road pricing. Format: yyyyMMddHHmmss

4.6.7.2 ChargingMessage

ChargingMessage is sent before the actual deduction occurs. This message indicates that a charge is about to be applied and is always followed by either a DeductionSuccessfulMessage or DeductionFailureMessage.

```
public struct ChargingMessage: Equatable {
    public let businessFunction: OBUBusinessFunction
    public let chargingType: OBUChargingType?
    public let chargingAmount: Int
}
```

Field	Type	Description
businessFunction	OBUBusinessFunction	One of the following: ERP, EPS, EEP, OPC, CPT, REP
chargingType	OBUChargingType?	To identify the ERP type (Point-Based or ERP1)
chargingAmount	Int	Charging amount in cents

4.6.7.3 DeductionSuccessfulMessage

DeductionSuccessfulMessage confirms that a payment deduction was successfully processed. This message includes the deducted amount and the payment mode used (frontend or backend).

```
public struct DeductionSuccessfulMessage: Equatable {
    public let businessFunction: OBUBusinessFunction
    public let chargingType: OBUChargingType?
    public let chargingAmount: Int
    public let paymentMode: OBUChargingPaymentMode
}
```

Field	Type	Description
businessFunction	OBUBusinessFunction	ERP, EPS, EEP OPC, CPT, REP
chargingType	OBUChargingType?	To identify the ERP type (Point-Based or ERP1)
chargingAmount	Int	Charging amount in cents
paymentMode	OBUChargingPaymentMode	The payment method used for the deduction either frontend (CEPAS card) or backend

4.6.7.4 DeductionFailureMessage

DeductionFailureMessage indicates that a payment deduction has failed. This message includes the card status, a descriptive failure reason, and the payment mode used for the deduction.

```
public struct DeductionFailureMessage: Equatable {
    public let businessFunction: OBUBusinessFunction
    public let chargingType: OBUChargingType?
    public let chargingAmount: Int
    public let paymentMode: OBUChargingPaymentMode
    public let cardStatus: OBUCardStatus
    public let failureMessage: String
}
```

Field	Type	Description
businessFunction	OBUBusinessFunction	ERP, EPS, EEP, OPC, CPT, REP
chargingType	OBUChargingType?	To identify the ERP type (Point-Based or ERP1)
chargingAmount	Int	Charging amount in cents

paymentMode	OBUCHargingPaymentMode	The payment method used for the deduction: either frontend (CEPAS card) or backend.
cardStatus	OBUCardStatus	The card status indicating the reason for the deduction failure (e.g., insufficientStoredValue, blacklisted, expiredStoredValue). Refer to section 6.8.1 for the complete list of card statuses.
failureMessage	String	Description of the failure reason derived from the card status.

4.6.7.5 MigrationMessage

MigrationMessage notifies motorists that the commencement of CPT or OPC is approaching.

```
public struct MigrationMessage: Equatable {
    public let businessFunction: OBUBusinessFunction
    public let migrationMessage: String
}
```

Field	Type	Description
businessFunction	OBUBusinessFunction	OPC, CPT
migrationMessage	String	Migration message

4.6.7.6 CarParkMessage

CarParkMessage is sent from the EEP car park when the vehicle exits the gantry. The isFromParkingOwner flag indicates whether the message originates from the car park owner or the car park system.

```
public struct CarParkMessage: Equatable {
    public let businessFunction: OBUBusinessFunction
    public let content1: String?
    public let content2: String?
    public let content3: String?
    public let content4: String?
    public let content5: String?
    public let isFromParkingOwner: Bool
}
```

Field	Type	Description
businessFunction	OBUBusinessFunction	EEP
content1	String?	Message from the carpark
content2	String?	Message from the carpark
Content3	String?	Message from the carpark
Content4	String?	Message from the carpark
Content5	String?	Message from the carpark
isFromParkingOwner	Boolean	If true, the message originates from the car park owner; otherwise, from the car park system.

4.6.7.7 SchemeUpdateMessage

SchemeUpdateMessage notifies that the vehicle is regarded as OPC vehicle (or non-OPC vehicle) from now on to motorists.

```
public struct SchemeUpdateMessage: Equatable {
    public let businessFunction: OBUBusinessFunction
    public let content1: String
    public let content2: String
}
```

Field	Type	Description
businessFunction	OBUBusinessFunction	OPC
content1	String	The scheme update message
content2	String	The name of the updated scheme

4.6.7.8 CommonAlertMessage

CommonAlertMessage is a general alert message not specific to any charging or parking event. It contains up to two content fields for displaying informational messages to the driver.

```
public struct CommonAlertMessage: Equatable {
    public let businessFunction: OBUBusinessFunction
    public let content1: String?
    public let content2: String?
}
```

Field	Type	Description
businessFunction	OBUBusinessFunction	ERP
content1	String?	First message in the alert (e.g. road name)
content2	String?	Second line of alert message (e.g., "Paya Lebar Rd")

4.6.7.9 ParkingStatusMessage

ParkingStatusMessage provides the current parking session status, including the car park location, session start time, and accumulated parking fee.

```
public struct ParkingStatusMessage: Equatable {
    public let businessFunction: OBUBusinessFunction
    public let carParkId: String
    public let carParkName: String
    public let startTime: String
    public let parkingFee: Int
}
```

Field	Type	Description
businessFunction	OBUBusinessFunction	REP
carParkId	String	Car park identifier
carParkName	String	Car park name
startTime	String	Parking session start time
parkingFee	Int	Current parking fee in cents

4.6.8 Migration Guide from OBUCargingInformation to ERPMessage

The SDK now uses a type-safe ERPMessage sealed class in place of the generic OBUCargingInformation and OBUCargingMessageType APIs. Each message type is a distinct subclass with its own strongly-typed fields, enabling compile-time safety and clearer code. This release also introduces five new message types such as MigrationMessage, SchemeUpdateMessage and ParkingStatusMessage, bringing the total to 12 messages.

Step 1: Implement the new delegate method

```
// Before
func onErpChargingInformation(_ chargingInfo: [OBUCargingInformation]?) {
}

// After
func onERPMessage(_ erpMessages: [ERPMessage]) {
}
```

```
// Before
func onErpChargingAndTrafficInfo(trafficInfo: OBUTrafficInfo?, chargingInfo:
[OBUCargingInformation]?) {
```

```
}

// After
func onERPMessageAndTrafficInfo(trafficInfo: OBUTrafficInfo?, erpMessages:
[ERPMessage]?) {
}
```

Step 2: Migrate from OBUChargingInformation to ERPMessage

Previously, message handling required checking the `chargingMessageType` enum on a generic `OBUChargingInformation` object. Now, `ERPMessage` is an enum with 12 specific cases. Use Swift's switch statement with pattern matching to handle each message type and access its associated data directly.

Before

```
for info in chargingList {
    switch info.chargingMessageType {
    case .alertPoint:
        let amount = info.chargingAmount
        let content = info.content1
    case .charging:
        // ...
    case .deductionSuccessful:
        // ...
    default:
        break
    }
}
```

After

```
for message in messages {
    switch message {
    case .alertPoint(let data):
        data.chargingAmount // type-safe access
        data.content1
    case .charging(let data):
        // ...
    case .deductionSuccessful(let data):
        // ...
    default:
        break
    }
}
```

Type Mapping Reference

Old ChargingMessageType	New ERPMessage Type
.alertPoint	.alertPoint(AlertPointMessage)

.charging	.charging(ChargingMessage)
.deductionSuccessful	.deductionSuccessful(DeductionSuccessfulMessage)
.deductionFailure	.deductionFailure(DeductionFailureMessage)
.cpoInformation	.carPark(CarParkMessage)
.detectAlertPointCommonAlert	.commonAlert(CommonAlertMessage)

Note: Since Common Alert is now a distinct message, all references `OBUSChargingType.Common` should be replaced with `ERPMessage.CommonAlertMessage`.

4.6.9 Code Migration Examples

4.6.9.1 Example 1: Alert Point Messages

Before: using `OBUSChargingInformation`

```
func handleAlertPoint(_ data: OBUSChargingInformation) {
    let location = data.content1 ?? ""
    let amount = data.chargingAmount
    let chargingType = data.chargingType

    let dollars = Double(amount) / 100
    print("Approaching \(location) - $\(dollars)")

    // Check charging type
    switch chargingType {
    case .pointBased:
        print("Point-based charging")
    default:
        break
    }
}
```

After: using `ERPMessage.AlertPointMessage`

```
func handleAlertPoint(_ data: AlertPointMessage) {
    let location = data.content1
    let amount = data.chargingAmount
    let chargingType = data.chargingType

    let dollars = Double(amount) / 100
    print("Approaching \(location) - $\(dollars)")
    print("Active: \(data.startTime) - \(data.endTime)")

    // Check charging type
    switch chargingType {
    case .pointBased:
        print("Point-based charging")
    default:
        break
    }
}
```

```

    }

    // Display alert to user
    showAlert(
        title: "ERP Alert",
        message: "Approaching \$(location)\nCharge: \$\$(String(format: "%.2f", dollars))"
    )
}

```

4.6.9.2 Example 2: Deduction Success Messages

Before: using OBUChargingInformation

```

func handleDeductionSuccess(_ data: OBUChargingInformation) {
    let amount = data.chargingAmount
    let paymentMode = data.paymentMode

    let dollars = Double(amount) / 100
    let mode = paymentMode == .frontend ? "Card" : "Account"

    print("✓ Payment successful: \$\$(dollars)")
    print("Payment method: \$(mode)")
}

```

After: using ERPMessage.DeductionSuccessfulMessage

```

func handleDeductionSuccess(_ data: DeductionSuccessfulMessage) {
    let amount = data.chargingAmount
    let paymentMode = data.paymentMode

    let dollars = Double(amount) / 100
    let mode = paymentMode == .frontend ? "Card" : "Account"

    print("✓ Payment successful: \$\$(dollars)")
    print("Payment method: \$(mode)")
}

```

4.6.10 SDK Helper Methods for OBU Data

1	func getCardBalance() -> Int?
	This function returns the last card balance if available (cents).
2	func getPaymentMode() -> OBUPaymentMode?
	This function returns the payment mode if available.

3	func getCardStatus() -> OBUCardStatus?
	This function returns the card status if available.
4	func getPaymentHistories() -> [OBUPaymentHistory]?
	This function returns the payment histories.
5	func getLastObuData() -> OBUData?
	This function returns the last received OBU Data. This class contains all the OBU data. It can be used to get complete data in one method.
6	func getPairedDevices(_ completion: @escaping Handler<[OBU]?>)
	This function returns a list of OBUs that have been previously paired with the smartphone.
7	func isSDKInitialised() -> Bool
	Returns the SDK initialised status.
8	func isDataPermissionGranted() -> Bool
	Returns the OBU Data permission status.
9	func getLastTrafficData() -> OBUTrafficData?
	Returns the last received traffic info data.
10	func getLastTravelSummary() -> OBUTravelSummary?
	Returns the last received Travel Summary data.

4.6.11 Mock Manager for iOS Applications

4.6.11.1 Overview

The OBU SDK includes a built-in Mock Manager that allows developers to simulate ERP charging messages and traffic information (TDCID) events without connecting to a physical OBU device. This is useful for development, testing, and demonstration purposes.

The Mock Manager uses three key classes:

- **MockSetting:** configures the time interval, card balance, event sequence and cyclic mode
- **MockEvent:** a container that holds one or more **ERPEvent** and an optional **TdcidEvent**
- **ERPEvent / TdcidEvent:** individual mock events for ERP messages and traffic information respectively

4.6.11.2 MockSetting

MockSetting is the configuration class used to control mock behaviour. It is accessed through the SDK's mock initialization.

Property	Type	Default	Description
timeInterval	Int	5	Time interval (in seconds) between each mock event in the sequence
cardBalance	Int	500	Simulated card balance (in cents) reflected in mock messages
sequence	[MockEvent]	Default mock sequence	Ordered list of mock events
cyclicMode	Bool	false	When enabled, the sequence repeats continuously from the beginning after the last event

4.6.11.3 MockEvent

MockEvent is a container that represents a single step in the mock sequence. Each **MockEvent** can contain multiple ERP messages and an optional traffic information event message, which are delivered together at each time interval.

MockEvent only requires ERP messages and/or TDCID events to be configured. All other data callbacks (velocity, acceleration, card information, etc.) are automatically generated by the mock and cannot be set manually.

Property	Type	Description
erpEvents	[ERPEvent]?	One or more ERP mock events to be delivered in this step
tdcidEvent	TdcidEvent?	An optional traffic information message to be delivered alongside the ERP events.

You can create a MockEvent with both ERP and TDCID events, or with just one of them:

```
// ERP + TDCID
MockEvent(erpEvents: [AlertPointMessage.Mock.erpPointBased()], tdcidEvent: .template1A)

// ERP only
MockEvent(erpEvents: [ChargingMessage.Mock.erpPointBased()])

// TDCID only
MockEvent(tdcidEvent: .template2B)
```

4.6.11.4 ERPEvent

Each ERPMessage type provides static mock factory methods through a nested Mock struct. These return ERPEvent objects with pre-configured values. Most methods accept optional parameters like chargingAmount, content1, paymentMode, and cardStatus.

Message Type	Mocks Methods	Params (Optional)
AlertPoint	erpPointBased, opcSingleArea, opcMultiArea, cpt	chargingAmount: Int, content1: String *content2: String (only for opcMultiArea)
Charging	erpPointBased, erp1, opc, rep, cpt, eep, eps	chargingAmount: Int
DeductionSuccessful	erpPointBased, erp1, eep, eps, opc, rep, cpt	chargingAmount: Int, paymentMode: OBUChargingPaymentMode
DeductionFailure	erpPointBased, erp1, eep, eps, opc, rep, cpt	chargingAmount: Int, paymentMode: OBUChargingPaymentMode

CommonAlert	erp	content1: String, content2: String
CarPark	ownerMessage, eeplInformation	content1: String, content2: String, content3: String, content4: String, *content5: String (only for eeplInformation)
SchemeUpdate	opc	content1: String, content2: String
ParkingStatus	parkingStatus	carParkId: String, carParkName: String, parkingFee: Int
Migration	cpt, opc	migrationMessage: String

Example:

```
AlertPointMessage.Mock.erpPointBased(chargingAmount: 200, content1: "Ophir Road")
DeductionFailureMessage.Mock.erpPointBased(chargingAmount: 100, paymentMode: .frontend,
cardStatus: .insufficientStoredValue)
```

4.6.11.5 TdcidEvent

TdcidEvent is a sealed class that provides pre-defined traffic information display templates. Each subclass represents a specific TDCID screen layout.

Template Range	Screen ID	Description
template1A – template1V	Screen 1	For unplanned event
template2A – template2E	Screen 2	
template3A – template3C	Screen 3	For free text layout
template4A – template4B	Screen 4	
template5A – template5B	Screen 5	
template6A – template6B	Screen 6	
template8A – template8Q	Screen 8	
template110A – template110B	Screen 110	For parking guidance system
template111A – template111B	Screen 111	For travel time display
template12A – template12R	Screen 12	For preloaded message
template13A	Screen 13	For speed limit

4.6.11.6 Setting Up a Mock Sequence

```
let mockSetting = MockBuilder()
    .setCardBalance(5000)
    .setTimeInterval(5)
    .setSequence([
        MockEvent(
            erpEvents: [AlertPointMessage.Mock.erpPointBased()],
            tdcidEvent: .template1B
        ),
        MockEvent(
            erpEvents: [ChargingMessage.Mock.erpPointBased()],
            tdcidEvent: .template2B
        ),
        MockEvent(
            erpEvents: [DeductionSuccessfulMessage.Mock.erpPointBased()],
            tdcidEvent: .template2C
        )
    ])
    .setCyclicMode(false)
    .build()
```

4.6.11.7 Simulating Concurrent Messages

When charging segments are close together (e.g. charging and deduction) on the road, the OBU may deliver multiple messages at the same time. To simulate this, add multiple ERPEvents to a single MockEvent.

```
let event = MockEvent(erpEvents: [
    AlertPointMessage.Mock.erpPointBased(chargingAmount: 200),
    ChargingMessage.Mock.erpPointBased(chargingAmount: 200)
], tdcidEvent: .template1A)
```

4.6.12 Traffic Icons in the OBU SDK

The SDK includes a bundle of icons specifically designed for Traffic messages, which can be utilised by applications. Each TrafficInfo message is associated with an icon that corresponds to a unique icon ID within the SDK. Refer to Appendix A for traffic icon list. The SDK has the bundle ID 'sg.gov.lta.extol' and the bundle name 'TDCID'.

```
if let bundle = Bundle(identifier: "sg.gov.lta.extol")?.path(forResource: "TDCID",
ofType: "bundle") {
    if let imagePath = Bundle(path: bundle)?.path(forResource: "1402",
ofType: "png") {
        print("image:\(imagePath)")
        iconView.image = UIImage.init(contentsOfFile: imagePath)
    }
}
```

5 MOCK MANAGER FOR TESTING APPLICATIONS

The SDK includes a Mock Manager for testing applications without a physical OBU. It provides APIs to generate mock data that replicates the messages sent from the OBU. The Mock Manager supports the following message types:

1. Electronic Road Pricing Messages (ERP)
2. Electronic Enhanced Parking (EEP)
3. Electronic Parking System (EPS)
4. Roadside Electronic Parking (REP)
5. Off-Peak Car (OPC)
6. Check Point Tolling (CPT)
7. Traffic Messages

For details on the Mock Manager, refer to [section 3.8.5 \(Android\)](#) and [section 4.6.11 \(iOS\)](#). The section below outlines the traffic messages that Mock Manager supports.

5.1 Traffic Messages

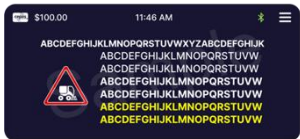
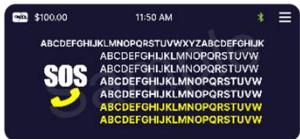
The Traffic messages/events are divided into three types:

- a) Traffic Text
- b) Travel Time
- c) Parking lots information.

The screenshots of these events are shown below to provide an example of how the information for these messages could be used in your application. The messages are represented in the class `OBUTrafficInfo`. It contains an attribute 'templateId' which represents the type of messages. The list below shows a type of messages that are used with the Mock Manager.

5.1.1 Template1

This template is defined as **unplannedIncidentsType1** and used to represent general traffic information. This belongs to type **Traffic Text**. The following are the list of templates and their corresponding messages and display layout.

Template1A 	Template1B 	Template1C 
Template1D	Template1E	Template1F



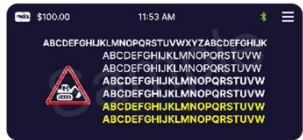
Template1G



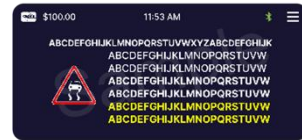
Template1H



Template1I



Template1J



Template1K



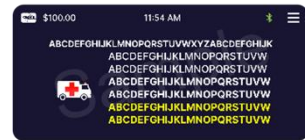
Template1L



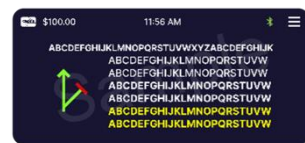
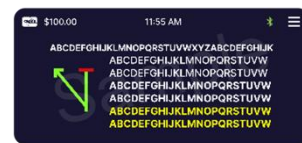
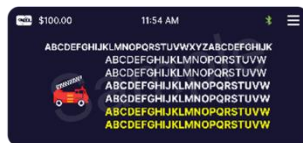
Template1M

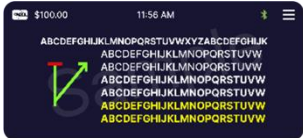
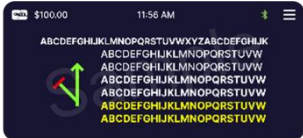
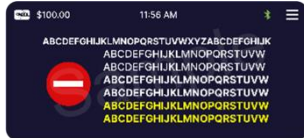
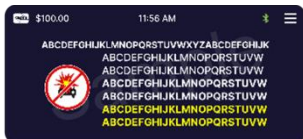
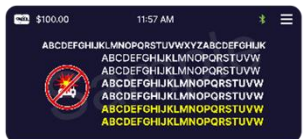
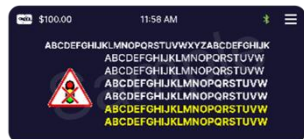
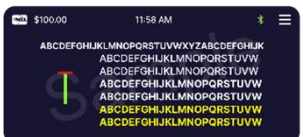


Template1N



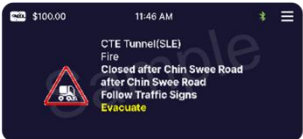
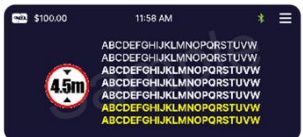
Template1O



Template1P 	Template1Q 	Template1R 
Template1S 	Template1T 	Template1U 
Template1V 		

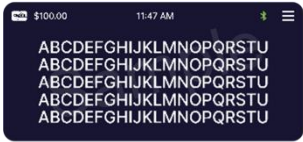
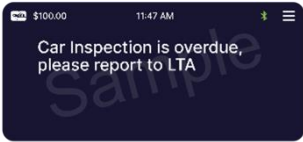
5.1.2 Template2

This template is defined as **unplannedIncidentsType11** and used to represent general traffic information. This belongs to type **Traffic Text**. The following are the list of templates and their corresponding messages and display layout.

Template2A 	Template2B 	Template2C 
Template2D 	Template2E 	

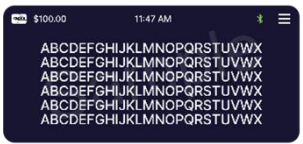
5.1.3 Template3

This template is defined as **freeText5Line** and used to represent general traffic information. This belongs to type **Traffic Text**. There can be maximum five lines. The following are the list of templates and their corresponding messages and display layout.:

Template3A	Template3B	Template3C
		

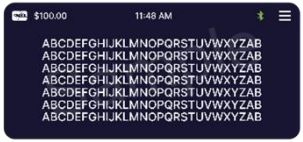
5.1.4 Template4

This template is defined as **freeText6Line** and used to represent general traffic information. This belongs to type **Traffic Text**. There can be maximum six lines. The following are the list of templates and their corresponding messages and display layout.

Template4A	Template4B
	

5.1.5 Template5

This template is defined as **freeText7Line** and used to represent general traffic information. This belongs to type **Traffic Text**. There can be maximum seven lines. The following are the list of templates and their corresponding messages and display layout.

Template5A	Template5B
	

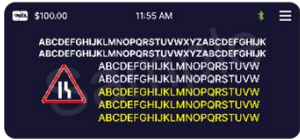
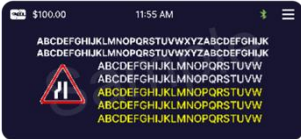
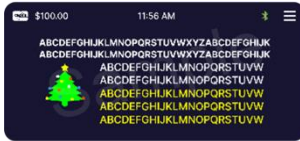
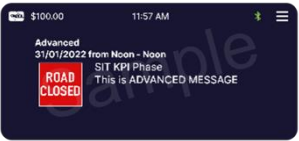
5.1.6 Template6

This template is defined as **freeText8Line** and used to represent general traffic information. This belongs to type **Traffic Text**. This template can display a maximum eight lines. The following are the list of templates and their corresponding messages and display layout.

5.1.7 Template8

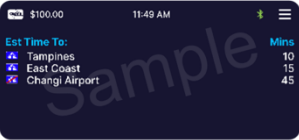
This template is defined as **plannedIncidents** and used to represent traffic information. This belongs to type **Traffic Text**. The following are the list of templates and their corresponding messages and display layout.

Template8A	Template8B	Template8C
Template8D	Template8E	Template8F
Template8G	Template8H	Template8I
Template8J	Template8K	Template8L

Template8M 	Template8N 	Template8O 
Template8P 	Template8Q 	

5.1.8 Template110

This **traveltime** template provides the information of the estimated time required to reach the destinations. The following are the list of templates and their corresponding messages and display layout.

Template110A 	Template110B 
---	---

5.1.9 Template111

This parking template defines and provides the information for the available parking lots. The following are the list of templates and their corresponding messages and display layout.

Template111A	Template111B
---------------------	---------------------

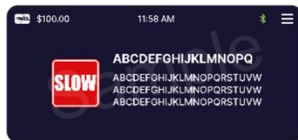


5.1.10 Template12

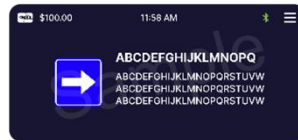
This template is defined as **preloadedData**. This belongs to type **Traffic Text**. The following are the list of templates and their corresponding messages and display layout.

Template12A	Template12B	Template12C
Template12D	Template12E	Template12F
Template12G	Template12H	Template12I
Template12J	Template12K	Template12L
Template12M	Template12N	Template12O

Template12P



Template12Q



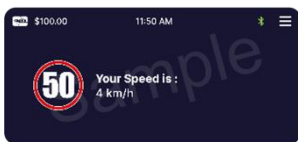
Template12R



5.1.11 Template13

This template is defined as **imageDisplay**. This belongs to type **Traffic Text**. The following are the list of templates and their corresponding messages and display layout.

Template13A



6 API CLASSES

These are the classes used in the APIs for iOS and Android.

6.1 OBU

This represents the OBU object containing the name of an OBU.

PROPERTIES	DESCRIPTION
name	String Name of the OBU.

6.2 OBUAcceleration

This set of parameters defines the acceleration data. The unit is m²/s.

PROPERTIES	DESCRIPTION
x	Float Acceleration of a vehicle along X axis
y	Float Acceleration of a vehicle along Y axis
z	Float Acceleration of a vehicle along Z axis

6.3 OBUPaymentHistory

This class represents payment history for successful charging.

PROPERTIES	DESCRIPTION
sequentialNumber	Int Id of the payment object
paymentDate	String? Represents date and time when the charging occurred. (format: YYYYMMDDHHMMSS)
businessFunction	OBUBusinessFunction Business function of the OBU (ERP/EPs/EEP/OPC/CPT/REP)
paymentMode	OBUChargingPaymentMode

	Charging Payment mode at the time of charging (Backend or Frontend)
chargeAmount	Int The charged amount after successful deduction, in cents

6.4 OBUTravelSummary

The OBU creates a Travel Summary when the vehicle's ignition is turned off. When the ignition turns on again, it sets the total travel distance of the previous trip and sent data to this class.

PROPERTIES	DESCRIPTION
totalTravelTime	Short Total Travel time covered in the last trip (Unit: Sec)
totalTravelDistance	Short Total travel distance covered in the last trip (Unit: 10m)

6.5 OBUTotalTripCharged

This class contains total charging information for all the business functions. The unit is cents.

PROPERTIES	DESCRIPTION
totalERP2charged	Int Total charge amount for ERP 2.0 from ignition ON to OFF.
totalEEPCharged	Int Total charge amount for EEP from ignition ON to OFF
totalREPCharged	Int Total charge amount for REP from ignition ON to OFF
totalOPCharged	Int Total charge amount for OPC from ignition ON to OFF
totalCPTCharged	Int Total charge amount for CPT from ignition ON to OFF

6.6 Traffic Information

Traffic messages are represented by the OBUTrafficInfo model. There are 3 types of traffic messages. The following are the variants:

1. TrafficParking: This represents parking information, i.e. number of parking spots available at different places.
2. TravelTime: It provides information about travel time to certain places.
3. TrafficText: It provides information about traffic related messages.

6.6.1 Usage in Android

In Android a sealed class is being used to provides the three types of Traffic messages.

```
sealed class OBUTrafficInfo<T>(  
    val priority: OBUTrafficDataPriority, val templateId: OBUTrafficMessageType,  
    val dataList: List<T>, val icon: OBUImageRef?  
) {  
  
    class OBUTrafficParking(  
        priority: OBUTrafficDataPriority,  
        templateId: OBUTrafficMessageType,  
        dataList: List<Parking>,  
        icon: OBUImageRef?  
    ) : OBUTrafficInfo<Parking>(priority, templateId, dataList, icon)  
  
    class OBUTravelTime(  
        priority: OBUTrafficDataPriority,  
        templateId: OBUTrafficMessageType,  
        dataList: List<TravelTime>,  
    ) : OBUTrafficInfo<TravelTime>(priority, templateId, dataList, icon)
```

```

        icon: OBUIImageRef?
    ) : OBUTrafficInfo<TravelTime>(priority, templateId, dataList, icon)

    class OBUTrafficText(
        priority: OBUTrafficDataPriority,
        templateId: OBUTrafficMessageType,
        dataList: List<OBUTextWithStyle>,
        icon: OBUIImageRef?
    ) : OBUTrafficInfo<OBUTextWithStyle>(priority, templateId, dataList, icon)

```

6.6.2 Usage in iOS

In iOS, an enum OBUTrafficInfo is used to send the three types of Traffic messages:

```

public protocol OBUBaseTraffic {
    associatedtype T
    var priority: OBUTrafficDataPriority { get set }
    var templateId: OBUTrafficMessageType { get set }
    var dataList: [T]? { get set }
}

public struct OBUTrafficParking: OBUBaseTraffic {
    public var priority: OBUTrafficDataPriority
    public var templateId: OBUTrafficMessageType
    public var dataList: [Parking]?
    public typealias T = Parking
}

public struct OBUTrafficTravelTime: OBUBaseTraffic {
    public var priority: OBUTrafficDataPriority
    public var templateId: OBUTrafficMessageType
    public var dataList: [TravelTime]?
    public typealias T = TravelTime
}

public struct OBUTrafficText: OBUBaseTraffic {
    public var priority: OBUTrafficDataPriority
    public var templateId: OBUTrafficMessageType
    public var dataList: [OBUTextWithStyle]?
    public typealias T = OBUTextWithStyle
    public let icon: OBUIImageRef?
}

@frozen public enum OBUTrafficInfo {
    case trafficText(OBUTrafficText)
    case trafficParking(OBUTrafficParking)
    case travelTime(OBUTrafficTravelTime)
}

```

The classes used in the OBUTrafficInfo are shown in tables below:

6.6.2.1 Parking

PROPERTIES	DESCRIPTION
location	OBUTextWithStyle Represents the parking lots name.
lots	OBUTextWithStyle Represents the number of lots available.

6.6.2.2 TravelTime

PROPERTIES	DESCRIPTION
location	OBUTextWithStyle Represents the destination name.
min	OBUTextWithStyle Travel time (in minutes) to the destination.
icon	OBUIImageRef It is a typealias of String and contains the icon name to be shown for the TDCID messages.

6.6.2.3 OBUTextWithStyle

The class provides the text information for each line in the Traffic info data.

PROPERTIES	DESCRIPTION
text	String Represents the data shown for TrafficInfo messages.
color	OBUTextColor Represent the colour of the text that the OBU display shows for each text for traffic messages
style	OBUTextStyle Represents the font weight of each text (whether its normal or Bold), as shown by the OBU Display

6.7 OBUData

This class contains all data points in one place.

PROPERTIES	DESCRIPTION
velocity	Double Velocity of the vehicle in m/s (range: 0 to 999.99)
acceleration	OBUAcceleration? Acceleration of the vehicle in m/s ² (range: -20.00 to 20.00)
erpMessages	List<ERPMessage>? A list of erp message
paymentHistories	List<OBUPaymentHistory>? Last five objects of charging payment history
cardBalance	Int? Balance of the CEPAS card
cardStatus	OBUCardStatus? Status of the CEPAS card. Refer to section 6.8.1 for the complete list of card statuses.
paymentMode	OBUPaymentMode? Payment mode of the OBU. Refer to section 6.8.5 for complete list of payment modes.
trafficInformation	OBUTrafficInfo? Traffic related messages
travelSummary	OBUTravelSummary? Travel Summary
totalCharged	OBUTotalTripCharged? Total Trip Charged

6.8 Enums

6.8.1 OBUCardStatus

VALUES	DESCRIPTION
Detected	Detected CEPAS card
Blacklisted	Blacklisted card
ExpiredStored	Expired stored value card
Invalid	Invalid card
IssuerIDError	Card Issuer ID Error
NoCardForFrontendPayment	No card or front-end payment mode
Blocked	Blocked card
FaultyStoreValue	Faulty stored value card
InsufficientStoredValue	Insufficient Stored Value
WrongStoredValueDebitCertificate	Wrong Stored Value Debit Certificate
NFCAccessError	NFC Access Error

6.8.2 OBUBusinessFunction

VALUES	DESCRIPTION
None	None
ERP	Electronic Road Pricing
EEP	Enhanced Electronic Parking
EPS	Electronic Parking System
REP	Road-side Electronic Parking
OPC	Off-Peak Car
CPT	Check Point Tolling

6.8.3 OBUCargingPaymentMode

VALUES	DESCRIPTION
--------	-------------

None	None
Frontend	OBU using CEPAS card for ERP
Backend	OBU using backend payment method enabled at e-Services portal.

6.8.4 OBUCargingType

VALUES	DESCRIPTION
None	None
PointBased	Point-based charging
Erp1	ERP1 message

6.8.5 OBUPaymentMode

VALUES	DESCRIPTION
Frontend	Inserted CEPAS card
Backend	Backend payment enabled at e-services portal
BusinessFunctionDisabled	No Business function is enabled

6.8.6 OBUTextColor

VALUES	DESCRIPTION
Default	Same as White
White	Represents colour of the text for traffic messages
Red	Represent colour of the text for traffic messages
Blue	Represents colour of the text for traffic messages on the OBU Display
Yellow	Represents colour of the text for traffic messages on the OBU Display
Green	Represents colour of the text for traffic messages on the OBU Display

Black	Represents colour of the text for traffic messages on the OBU Display
--------------	---

6.8.7 OBUTextStyle

VALUES	DESCRIPTION
Default	Same as normal style of the OBU display
Normal	Normal text
Bold	Bold text

6.8.8 OBUTrafficDataPriority

VALUES	DESCRIPTION
High	It represents emergency or high priority Traffic messages
Charging	ERP1/ERP/EEP/EPS/REP/OPC/CPT charging related messages
Medium	Messages related to Information and driver operations
Low	Low priority Traffic messages

6.8.9 OBUTrafficMessageType

VALUES	DESCRIPTION
UnplannedIncidentsTypeI	This type has seven texts and one image. The examples of it are shown in 5.1.1 Template1.
UnplannedIncidentsTypeII	This type has seven texts and one image. The examples of it are shown in 5.1.2 Template2.
FreeText5Line	This type has five texts. The examples of it are shown in 5.1.3 Template3.
FreeText6Line	This type has six texts. The examples of it are shown in 5.1.4 Template4.
FreeText7Line	This type has seven texts. The examples of it are shown in 5.1.5 Template5.
FreeText8Line	This type has eight texts. The examples of it are shown in 5.1.6 Template6.
PlannedIncidents	This type has seven texts and one image. The examples of it are shown in 5.1.7 Template8.
TravelTime	This type has travel time information for destinations. The examples of it are shown in 5.1.8 Template10.
ParkingGuidance	This type has parking availability information. The examples of it are shown in 5.1.9 Template11.
PreloadedData	This type has some texts and one image. The examples of it are shown in 5.1.10 Template12.
ImageDisplay	This type has one image. The examples of it are shown in 5.1.11 Template13.

6.8.10 BluetoothState (iOS)

It represents the current state of a CBManager.

VALUES	DESCRIPTION
unknown	State unknown, update imminent

resetting	The connection with the system service was momentarily lost, update imminent.
unsupported	The platform does not support the Bluetooth Low Energy role.
unauthorized	The application is not authorized to use the Bluetooth Low Energy role.
poweredOff	Bluetooth is currently powered off.
poweredOn	Bluetooth is currently powered on and available to use.

6.8.11 BluetoothState (Android)

This enum represents the states of 'ACTION_STATE_CHANGED' broadcast intent, which the system broadcasts whenever the Bluetooth state changes.

VALUES	DESCRIPTION
STATE_TURNING_ON	Indicates the local Bluetooth adapter is turning on.
STATE_ON	Indicates the local Bluetooth adapter is on, and ready for use.
STATE_TURNING_OFF	Indicates the local Bluetooth adapter is turning off.
STATE_OFF	Indicates the local Bluetooth adapter is off.

6.8.12 OBUErrorCode

VALUES	ERRORCODE	DESCRIPTION
INTERNET_NOT_AVAILABLE	1000	When there is no internet connectivity on the phone.
BLUETOOTH_DISABLED	1001	When the Bluetooth on the phone is disabled or turned off.
DEVELOPER_UNAUTHORISED	1003	When an SDK Account key issued to the developer is blocked.
APPLICATION_UNAUTHORISED	1004	When a specific application using an SDK Account issued to a developer is blocked.

DEVELOPER_DEACTIVATED	1005	In case if a developer is not allowed to use the SDK account key issued to them after registration at DataMall. You may contact EXTOL support team if this error persists.
SEARCH_ERROR (for Android)	1006	This is a specific error for Android related to companion device pairing scan. This is an internal error of that.
OBU_NOT_PAIED	2000	When the SDK tries to make a connection to the OBU which has not paired with the phone. This is specifically returned for the connect method via vehicle number.
CONNECTION_FAILED	2001	This is a generic error that occurs when connection with the OBU fails.
DEVICE_DATA_ACCESS_ERROR	2002	This represents an internal data error in the SDK while getting data from the OBU. Please contact EXTOL support team if this persists.
SDK_AUTHENTICATION_REQUIRED	2003	Calling the SDK APIs without authentication, throws this error code. The developer must initialise the SDK before calling the SDK methods.
OBU_DATA_ACCESS_PERMISSION_DENIED	2004	If OBU data permission is not taken from the motorist. The developer must ensure that this permission has been taken from the motorist in their applications. This is to inform the motorist about the OBU data being used by third-party applications.
CODE_SIGNATURE_FAIL	2009	This is an internal SDK data error. If this error persists, please contact EXTOL support team.
NO_PUBLIC_KEY_AVAILABLE	2010	This is an internal SDK data error. If this error persists, please contact customer support.
BAD_REQUEST	400	HTTP error code
INVALID_SDK_ACCOUNT_KEY	401	If the SDK account key is not correct
REQUEST_TIMEOUT	408	HTTP error code
SERVER_ERROR	500	HTTP error code

MOCK_SEQUENCE_ENDED (for Android)	3000	Returned when the mock manager sequence of events is terminated. This code can be used to find out the end of sequence. This is only used for the Mock Manager.
MAC_ADDRESS_AUTHENTICATION_FAILED (iOS)	2006	User is required to register their iPhone's Bluetooth MAC Address with e-Services. If Bluetooth MAC Address entered is not registered / incorrect and an iPhone tries to make the connection, this error is returned in that case. Upon receiving this error, you can inform the user to enter the MAC Address next time when the connection request is initiated. The Bluetooth MAC Address User Interface is presented by the SDK during the connection process. It is required only one time. Once entered by the user, the User Interface does not appear again.
TRANSPORT_ERROR (iOS)	616	This error is returned if the internet is connected but data packets cannot be sent. Only applicable for iOS.

7 SDK TROUBLESHOOTING

Here are some common issues you may come across when using the SDK and how to solve them.

How do I know the OBU is ready to pair with my smartphone?

After switching on the car, the OBU takes 1-2 mins to be ready to pair with your smartphone. The OBU Display shows a Bluetooth icon at the top right corner of the screen when it is ready to pair and connect. Absence of it can mean two things: either your smartphone Bluetooth MAC Address has not been registered or if it is registered, the OBU will receive it in a while and start showing the icon. After that, the OBU is ready to pair and connect to your phone.

I am not able to pair & connect my phone to the OBU?

There is one main reason this might be happening. Your phone is not registered with the e-services Portal. Without registration, the pairing and connection will not take place. Please refer to [section 2.3](#) explaining the registration process for iPhone and Android.

What to do with the error codes (OBUErrorCode) and how to fix it?

OBUErrorCode explains all the errors that you may receive during development and testing if something has gone wrong. If some error persists, please write to us with details of what you have tried at EXTOL@lta.gov.sg.

8 APPENDIX A - TDCID ICONS

125		1402		1411		1421		1545		1	
502		1403		1412		1422		1546		2	
520		1404		1413		1423		1701		9003	
521		1405		1414		1424		1702		9007	
527		1406		1415		1425		1705		9006	
528		1407		1416		1426		1706		9008	
529		1408		1417		1427		2001		9009	
530		1409		1418		1428		2002		9010	
1199		1410		1419		1540		2003		9011	
				1420		1541		2004			
						1542		2005			
						1543					